جزوه

Ops

عملیات: راه‌اندازی سرور، انتشار، استقرار، بکاپ و بازگردانی

1. A script that stops on the first error · اسکریپت ایمن: اولین خطا یعنی توقف
2. Setting up a fresh server · راه‌اندازی سرور تازه: SSH، فایروال، fail2ban و swap
3. A release: check, show the plan, ask, then ship · انتشار: اول چک، بعد نقشه، بعد تأیید، بعد ارسال
4. Deploying on the server: one at a time, fast-forward only, wait for health · استقرار روی سرور: یکی‌یکی، فقط fast-forward، صبر برای سلامت
5. The nightly job: backup, retention and a log · کار شبانه: بکاپ، نگه‌داری ۱۴ روزه و لاگ
6. A backup that was never restored is not a backup · بازگردانی: بکاپی که restore نشده، بکاپ نیست
7. CI/CD to a VPS: deploy after tests, with a key that can only deploy · CI/CD: استقرار بعد از تست، با کلیدی که فقط deploy بلد است
8. A reverse proxy in front of every app · reverse proxy: HTTPS خودکار، basic auth و هدرهای امنیتی

درس 1 · ops

A script that stops on the first error

اسکریپت ایمن: اولین خطا یعنی توقف

set -euo pipefail | ErrorActionPreference | exit code | fail fast

بعد از این درس می‌توانی

- بدانی Bash و PowerShell به‌طور پیش‌فرض بعد از خطا ادامه می‌دهند و Python نه، و هر کدام را چطور سخت‌گیر کنی.
- خطای یک pipeline، متغیر تعریف‌نشده و exit code برنامه‌های بیرونی را در هر سه زبان بگیری.
- جاهایی را بشناسی که حالت سخت‌گیر خودش اشتباه می‌کند: grep بدون نتیجه، head و SIGPIPE، و local.

سناریو از پروژه‌ی واقعی

همه‌ی اسکریپت‌های deploy/vps در Control Panel با set -euo pipefail شروع می‌شوند و release.ps1 با $ErrorActionPreference = 'Stop'. این یک خط تفاوت بین «بکاپ گرفته نشد و خبردار شدی» و «بکاپ گرفته نشد و لاگ نوشت done» است. نمونه‌ی این درس یک خلاصه‌ساز لاگ است، با یک نسخه‌ی عمداً غلط کنارش: unsafe-log-summary.sh به فایلی که وجود ندارد می‌گوید «errors: 0» و با کد ۰ تمام می‌شود. تست‌ها همین را هم چک می‌کنند.

سؤال چرا Bash بعد از یک دستور خراب ادامه می‌دهد؟

پاسخ پوسته برای کار تعاملی ساخته شده: در ترمینال نمی‌خواهی با هر خطا پنجره بسته شود. اسکریپت همان قاعده را به ارث می‌برد، مگر اینکه با set -e عوضش کنی.

سؤال PowerShell که exception دارد؛ مشکلش چیست؟

پاسخ بیشتر خطاهای cmdletها non-terminating هستند: پیام قرمز چاپ می‌شود و خط بعد اجرا می‌شود. 'Stop' آن‌ها را exception می‌کند. ولی برنامه‌های بیرونی مثل git یا sshd اصلاً exception نمی‌دهند؛ فقط $LASTEXITCODE عوض می‌شود.

مفهوم‌ها

سه حرف Bash

-e: اگر دستوری با کد غیر صفر تمام شد، اسکریپت هم تمام شود. -u: استفاده از متغیر تعریف‌نشده خطاست (rm -rf "/"$DIR با DIR خالی یعنی rm -rf /). -o pipefail: کد خروج یک pipeline کد آخرین دستور ناموفق آن است، نه فقط دستور آخر. بدون آن grep x missing.log | wc -l صفر چاپ می‌کند و موفق است.

PowerShell: دو نوع خطا

$ErrorActionPreference = 'Stop' خطاهای cmdletها را terminating می‌کند و Set-StrictMode -Version Latest متغیر تعریف‌نشده را خطا می‌کند. برای برنامه‌های native باید بعد از هر فراخوانی $LASTEXITCODE را چک کنی؛ از PowerShell 7.4 می‌شود $PSNativeCommandUseErrorActionPreference = $true را هم روشن کرد. release.ps1 این کار را در تابع Git انجام می‌دهد.

Python: پیش‌فرض درست، با یک استثنا

هر exception که گرفته نشود اسکریپت را با کد ۱ تمام می‌کند. استثنا subprocess.run است: بدون check=True کد خروج برنامه فقط در returncode می‌ماند و کسی نگاهش نمی‌کند.

وقتی سخت‌گیری اشتباه می‌کند

grep وقتی چیزی پیدا نکند کد ۱ می‌دهد، و با pipefail «خطایی نبود» می‌شود «اسکریپت مُرد». پس فقط کد ۱ را ببخش (|| [[$? -eq 1]])، نه همه را (|| true). head و grep -q زود می‌بندند و دستور قبلی با SIGPIPE (کد ۱۴۱) می‌میرد: تست درس دوم همین را در sshd -T | grep -qx گرفت. و local x=$(cmd) کد خروج cmd را می‌پوشاند، چون local خودش موفق است.

مقایسه

روشن کردن توقف روی خطا	
Bash	`set -euo pipefail`
PowerShell	`$ErrorActionPreference = 'Stop'`
Python	(پیش‌فرض)

فقط Python بدون یک خط اضافه درست رفتار می‌کند.

متغیر تعریف‌نشده	
Bash	`set -u`
PowerShell	`Set-StrictMode -Version Latest`
Python	`NameError` (پیش‌فرض)

بدون آن در Bash و PowerShell متغیر خالی بی‌صدا رشته‌ی خالی می‌شود.

خطای برنامه‌ی بیرونی	
Bash	خودکار -e با
PowerShell	`if ($LASTEXITCODE -ne 0) { throw }`
Python	`subprocess.run(..., check=True)`

'Stop' در PowerShell روی git و sshd اثری ندارد؛ این بزرگ‌ترین سوراخ آن است.

خطای وسط pipeline	
Bash	`set -o pipefail`
PowerShell	متوقف می‌کند 'Stop' با cmdlet خطای
Python	ندارد؛ هر قدم جدا pipeline

pipeline پاورشل شیء جابه‌جا می‌کند، نه متن؛ مشکل SIGPIPE را ندارد.

کد خروج	
Bash	`exit 1`
PowerShell	`exit 1` یا `throw` (کد ۱)
Python	`sys.exit(1)` یا `sys.exit('پیام')`

sys.exit با رشته پیام را در stderr می‌نویسد و کد ۱ برمی‌گرداند.

کد

log-summary: شمردن خطاهای لاگ

Bash کد اجراشده و تست‌شده در /examples examples/ops/safe/log-summary.sh

```bash
set -euo pipefail

log="${1:?usage: log-summary.sh <log file>}"

# grep exits 0 = found, 1 = nothing found, 2 = could not read the file. Under pipefail
# "nothing found" would stop the script, but here it is an answer: only 1 is excused.
matches() { grep "$@" || [[ $? -eq 1 ]]; }

count="$(matches -c 'ERROR' "$log")"
echo "errors: $count"
# awk, not head: head exits after 3 lines and a long sort output would then die of
# SIGPIPE, which pipefail reports as a failure (exit 141).
matches 'ERROR' "$log" | sed -E 's/.*ERROR:? *//' | sort | uniq -c | sort -rn | awk 'NR <= 3'
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/safe/log-summary.ps1

```powershell
param([Parameter(Mandatory)][string]$Log)
$ErrorActionPreference = 'Stop'
Set-StrictMode -Version Latest

# A missing file is a non-terminating error for Select-String: without 'Stop' it prints
# red text, $found is empty and the script reports 0. -CaseSensitive: grep is, and
# Select-String is not by default.
$found = @(Select-String -LiteralPath $Log -Pattern 'ERROR' -SimpleMatch -CaseSensitive)
"errors: $($found.Count)"
$found | ForEach-Object { $_.Line -replace '.*ERROR:? *', '' } |
    Group-Object -NoElement | Sort-Object Count -Descending | Select-Object -First 3 |
    ForEach-Object { '{0,7} {1}' -f $_.Count, $_.Name }
```

Python کد اجراشده و تست‌شده در /examples examples/ops/safe/log_summary.py

```python
import re
import sys
from collections import Counter
from pathlib import Path

# Python stops by itself: a missing file raises, and an uncaught exception ends the
# script with exit code 1 and a traceback. Nothing to switch on.
lines = Path(sys.argv[1]).read_text(encoding="utf-8").splitlines()
errors = [line for line in lines if "ERROR" in line]
print(f"errors: {len(errors)}")
messages = Counter(re.sub(r".*ERROR:? *", "", line, count=1) for line in errors)
for message, n in messages.most_common(3):
    print(f"{n:7d} {message}")
```

اشتباه‌های رایج

Bash **اسکریپت بدون set -euo pipefail**

چرا: unsafe-log-summary.sh به لاگی که وجود ندارد می‌گوید errors: 0 و موفق تمام می‌شود. cron هم چیزی گزارش نمی‌کند.

درست: set -euo pipefail خط دوم هر اسکریپت، بعد از shebang.

Bash **grep ... || true**

چرا: خطای واقعی (فایل نیست، کد ۲) هم با «پیدا نشد» (کد ۱) قورت داده می‌شود.

درست: grep ... || [[$? -eq 1]]

PowerShell **تکیه بر 'Stop' برای git، docker یا sshd**

چرا: برنامه‌ی native با کد غیر صفر exception نمی‌دهد؛ اسکریپت ادامه می‌دهد.

درست: بعد از هر فراخوانی $LASTEXITCODE را چک کن، یا یک تابع کوچک مثل Invoke-Git بساز.

PowerShell **Select-String به‌جای grep بدون -CaseSensitive**

چرا: Select-String پیش‌فرض حساس به حروف نیست؛ «error» کوچک هم شمرده می‌شود و عدد با grep فرق می‌کند.

درست: -CaseSensitive، و -SimpleMatch وقتی الگو regex نیست.

Python **subprocess.run بدون check=True**

چرا: برنامه‌ی ناموفق فقط returncode را عوض می‌کند و اسکریپت ادامه می‌دهد.

درست: check=True، یا چک صریح returncode.

این کد کجای پروژه‌هاست

`ControlPanel/deploy/vps/bootstrap.sh:20`

set -euo pipefail، با log و die برای پیام‌ها

`ControlPanel/deploy/vps/release.ps1:36`

'ErrorActionPreference = 'Stop$ و تابع Git که $LASTEXITCODE را چک می‌کند

`ControlPanel/deploy/vps/release.ps1:79`

git پیشرفت را در stderr می‌نویسد و Windows PowerShell 5.1 با 'Stop' آن را خطا می‌گیرد؛ تصمیم با exit code است

تمرین

▼ 1. در log-summary.sh خط count=... را به local count=... داخل یک تابع تبدیل کن و فایل ناموجود بده. چه می‌شود؟

راهنما: کد خروج local چیست؟

پاسخ: اسکریپت ادامه می‌دهد و «errors: » خالی چاپ می‌کند: local موفق است و کد خروج command substitution را می‌پوشاند. درستش: local count; count="$(...)".

▼ 2. چرا در log-summary.sh به‌جای head -3 از 'awk 'NR <= 3' استفاده شده؟

پاسخ: head بعد از سه خط pipe را می‌بندد. اگر sort هنوز در حال نوشتن باشد SIGPIPE می‌گیرد و با کد ۱۴۱ تمام می‌شود، و pipefail کل اسکریپت را ناموفق می‌کند. awk همه‌ی ورودی را می‌خواند.

▼ 3. در PowerShell بنویس: git pull و اگر ناموفق بود اسکریپت با پیام متوقف شود.

پاسخ: git pull --ff-only; if ($LASTEXITCODE -ne 0) { throw 'git pull failed' } یا در 7.4 به بعد: $PSNativeCommandUseErrorActionPreference = $true قبل از آن.

جمله‌ی مصاحبه

Every script starts strict: set -euo pipefail in Bash, ErrorActionPreference Stop plus an exit-code check after every native command in PowerShell, check=True for subprocess in Python, and the only error I excuse is the one I name, like grep's exit code 1.

درس 2 · ops

Setting up a fresh server

راه‌اندازی سرور تازه: SSH، فایروال، fail2ban و swap

non-root user | swap | fail2ban | ufw | ssh hardening

بعد از این درس می‌توانی

- یک سرور Ubuntu تازه را به کاربر غیر root، ورود فقط با کلید، فایروال، fail2ban، به‌روزرسانی خودکار و swap برسانی.
- بفهمی چرا ترتیب نام فایل‌های sshd_config.d مهم است و تنظیم را از خود sshd بپرسی، نه از فایل.
- کاری کنی که اشتباه در کلید، تو را از سرور بیرون نیندازد.

سناریو از پروژه‌ی واقعی

bootstrap.sh یک بار روی VPS با sudo اجرا شد. سخت‌ترین قسمتش یک خط بود: فایل سخت‌سازی اول 99-hardening.conf نام داشت و sshd هنوز رمز عبور قبول می‌کرد. دلیلش: image ابری OVH فایل 50-cloud-init.conf با PasswordAuthentication yes دارد، و sshd برای هر تنظیم اولین مقداری را که می‌خواند نگه می‌دارد. حالا فایل 00-hardening.conf است و اسکریپت با sshd -T از خود sshd می‌پرسد. تست این درس همان وضعیت را در یک کانتینر Ubuntu 24.04 می‌سازد و نشان می‌دهد 99- می‌بازد.

سؤال چرا کاربر deploy، وقتی root هست؟

پاسخ root را همه‌ی رباتهای اینترنت امتحان می‌کنند و هر اشتباهی با آن بی‌حد است. deploy با sudo کار می‌کند، و ورود مستقیم root از SSH بسته است.

سؤال چرا اسکریپت می‌گوید session اول را نبند؟

پاسخ اگر کلید درست کپی نشده باشد، بعد از بسته شدن ورود با رمز دیگر راهی به داخل نیست. session باز راه برگشت است تا وقتی ssh deploy@server از ترمینال دوم کار کند.

مفهوم‌ها

SSH: اولین مقدار برنده است

sshd_config در اولین خطش Include /etc/ssh/sshd_config.d/*.conf دارد و فایل‌ها به ترتیب الفبا خوانده می‌شوند. برای بیشتر تنظیم‌ها اولین مقدار می‌ماند. پس فایل سخت‌سازی باید قبل از 50-cloud-init.conf بیاید. sshd -t نحو را چک می‌کند و sshd -T تنظیم نهایی را چاپ می‌کند.

فایروال و Docker

ufw: همه‌ی ورودی‌ها بسته، فقط 22، 80 و 443. ولی Docker پورت‌های publishشده را مستقیم در iptables باز می‌کند و ufw آن را نمی‌بیند. برای همین compose.vps.yml همه‌ی ports را با !reset برمی‌دارد و فقط Caddy پورت دارد.

fail2ban و به‌روزرسانی خودکار

fail2ban لاگ sshd را می‌خواند و IPای که ۵ بار در ۱۰ دقیقه شکست خورد را یک ساعت می‌بندد. unattended-upgrades وصله‌های امنیتی را هر روز خودش نصب می‌کند؛ ریبوت با توست.

swap و لاگ کانتینرها

SQL Server و buildها حافظه می‌خورند؛ ۴ گیگ swap جلوی OOM killer را در اوج می‌گیرد و vm.swappiness=10 یعنی فقط در اضطرار. daemon.json لاگ کانتینرها را به ۳ فایل ۱۰ مگابایتی محدود می‌کند، وگرنه یک روز دیسک پر می‌شود.

در ویندوز چه معادلی دارد؟

OpenSSH Server ویندوز همان sshd_config را دارد (C:\ProgramData\ssh)، فایروال Windows Defender با New-NetFirewallRule، و به‌جای fail2ban معمولاً قفل حساب در Local Security Policy. PowerShell 7 روی لینوکس هم اجرا می‌شود، و نسخه‌ی PowerShell این درس در همان کانتینر Ubuntu تست شده.

مقایسه

نوشتن فایل تنظیم

Bash	`cat > file <<'EOF' ... EOF`
PowerShell	`@(...) -join "`n" | Set-Content -NoNewline`
Python	`Path(file).write_text(text)`

here-string پاورشل پایان خط خود فایل .ps1 را می‌گیرد؛ checkout با CRLF به «sshd\r no» می‌دهد.

پاک کردن فایلی که شاید نباشد

Bash	`rm -f file`
PowerShell	`Remove-Item file -ErrorAction Ignore`
Python	`Path(file).unlink(missing_ok=True)`

بدون -Stop، 'ErrorAction Ignore' نبودن فایل را خطا می‌کند.

پرسیدن تنظیم واقعی

Bash	`sshd -T | grep -qx 'passwordauthentication no'`
PowerShell	`(sshd -T) -notcontains 'passwordauthentication no'`
Python	`'...' in run(['sshd','-T']).stdout.splitlines()`

خروجی native در PowerShell آرایه‌ای از خط‌هاست؛ -contains کل خط را مقایسه می‌کند.

اجرا با دسترسی root

Bash	`[[ $EUID -eq 0 ]] || die`
PowerShell	`[Environment]::UserName -eq 'root'  (ویندوز: #Requires -RunAsAdministrator)`
Python	`os.geteuid() == 0`

#Requires -RunAsAdministrator فقط در ویندوز معنی دارد.

کد

سخت‌سازی SSH و اثباتش

Bash کد اجراشده و تست‌شده در /examples examples/ops/server/harden-ssh.sh

```
# sshd keeps the FIRST value it reads for a setting, and cloud images ship
# 50-cloud-init.conf with "PasswordAuthentication yes": this file must sort before it.
rm -f "$DIR/99-hardening.conf"
cat > "$DIR/00-hardening.conf" <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
X11Forwarding no
EOF

sshd -t   # syntax check: a broken config would lock everyone out on the next reload
# Ask sshd what it will do, not what the file says. Into a variable first: in
# `sshd -T | grep -q`, grep quits at the first match, sshd dies of SIGPIPE writing
# the rest, and pipefail reports the whole pipeline as failed.
effective="$(sshd -T)"
grep -qx 'passwordauthentication no' <<<"$effective" || { echo "sshd still accepts passwords" >&2; exit 1; }
echo "ssh hardened"
```

PowerShell کد اجراشده و تست‌شده در /examples `examples/ops/server/harden-ssh.ps1`

```powershell
# sshd keeps the FIRST value it reads for a setting, and cloud images ship
# 50-cloud-init.conf with "PasswordAuthentication yes": this file must sort before it.
Remove-Item "$dir/99-hardening.conf" -ErrorAction Ignore
# Joined with "`n": a here-string takes the line endings of the .ps1 file itself,
# and a CRLF checkout would hand sshd "no`r".
@(
    'PermitRootLogin no'
    'PasswordAuthentication no'
    'KbdInteractiveAuthentication no'
    'PubkeyAuthentication yes'
    'MaxAuthTries 3'
    'X11Forwarding no'
) -join "`n" | Set-Content -NoNewline "$dir/00-hardening.conf"

# 'Stop' covers cmdlets only: a native program that fails just sets $LASTEXITCODE.
sshd -t
if ($LASTEXITCODE -ne 0) { throw 'sshd -t rejected the configuration' }
$effective = sshd -T
if ($LASTEXITCODE -ne 0) { throw 'sshd -T failed' }
if ($effective -notcontains 'passwordauthentication no') { throw 'sshd still accepts passwords' }
'ssh hardened'
```

Python کد اجراشده و تست‌شده در /examples `examples/ops/server/harden_ssh.py`

```python
HARDENING = """\
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
X11Forwarding no
"""

# sshd keeps the FIRST value it reads for a setting, and cloud images ship
# 50-cloud-init.conf with "PasswordAuthentication yes": this file must sort before it.
(DIR / "99-hardening.conf").unlink(missing_ok=True)
(DIR / "00-hardening.conf").write_text(HARDENING, encoding="utf-8")

# check=True turns a non-zero exit into an exception; without it, a failure is silent.
subprocess.run(["sshd", "-t"], check=True)
effective = subprocess.run(["sshd", "-T"], check=True, capture_output=True, text=True).stdout
if "passwordauthentication no" not in effective.splitlines():
    sys.exit("sshd still accepts passwords")
print("ssh hardened")
```

بقیه‌ی bootstrap.sh (Bash): فایروال، swap، fail2ban

همه‌ی زبان‌ها از کد واقعی پروژه (کوتاه‌شده)

```
ufw default deny incoming
ufw default allow outgoing
ufw allow OpenSSH
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow 443/udp   # HTTP/3
ufw --force enable

cat > /etc/fail2ban/jail.d/sshd.local <<'EOF'
[sshd]
enabled = true
maxretry = 5
findtime = 10m
bantime = 1h
EOF
systemctl enable --now fail2ban

if ! swapon --show | grep -q .; then   # only once: the script can run again
  fallocate -l "$SWAP_SIZE" /swapfile
  chmod 600 /swapfile
  mkswap /swapfile
  swapon /swapfile
  echo '/swapfile none swap sw 0 0' >> /etc/fstab
  echo 'vm.swappiness=10' > /etc/sysctl.d/99-swappiness.conf
fi
```

اشتباه‌های رایج

همه **نام 99-hardening.conf**

چرا: 50-cloud-init.conf زودتر خوانده می‌شود و PasswordAuthentication yes آن می‌ماند. تست این درس این را نشان می‌دهد.

درست: 00-hardening.conf، و همیشه sshd -T برای اثبات.

همه **بستن session اول قبل از امتحان کاربر جدید**

چرا: کلید اشتباه یعنی دیگر هیچ راهی به سرور نیست جز کنسول ارائه‌دهنده.

درست: از ترمینال دوم ssh deploy@server؛ فقط بعد از موفقیت، اولی را ببند.

همه **اعتماد به ufw برای پورت‌های Docker**

چرا: "ports: "1433:1433 در compose از ufw رد می‌شود و SQL Server به اینترنت باز است.

درست: هیچ پورتی جز reverse proxy publish نکن، یا به 127.0.0.1 ببند.

همه **اسکریپتی که دوبار اجرا شدنش خراب می‌کند**

چرا: دومین اجرا خط swap را دوباره به fstab اضافه می‌کند یا کاربر را دوباره می‌سازد و خطا می‌دهد.

درست: هر قدم اول چک کند: if ! swapon --show | grep -q، if ! id user .

Bash **sshd -T | grep -qx ... زیر pipefail**

چرا: grep -q با اولین تطابق خارج می‌شود، sshd در نوشتن بقیه‌ی خروجی SIGPIPE می‌گیرد و pipefail کل pipeline را شکست می‌داند. نتیجه تصادفی است: بسته به اینکه خط کجای خروجی باشد. تست این درس اول همین را گرفت، و bootstrap.sh هم همین خط را داشت (حالا درست شده).

درست: اول در متغیر: "effective="$(sshd -T)"، بعد "<<<"$effective ... grep -qx".

PowerShell **here-string برای فایل تنظیم لینوکس**

چرا: اگر .ps1 با CRLF ذخیره شده باشد، هر خط به r\ ختم می‌شود و sshd آن را نمی‌فهمد.

درست: خط‌ها را با "`n" join کن، یا .gitattributes با eol=lf.

این کد کجای پروژه‌هاست

`ControlPanel/deploy/vps/bootstrap.sh:62`

چرا 00- و نه 99-: sshd اولین مقدار را نگه می‌دارد

`ControlPanel/deploy/vps/bootstrap.sh:76`

sshd -T در یک متغیر، بعد grep: اثبات اینکه رمز عبور واقعاً بسته است، بدون شکست تصادفی SIGPIPE

`ControlPanel/deploy/vps/bootstrap.sh:89`

Docker از ufw رد می‌شود؛ فقط پورت‌های Caddy publish شده‌اند

`ControlPanel/deploy/vps/bootstrap.sh:132`

محدود کردن لاگ کانتینرها در daemon.json

تمرین

▼ 1. روی سرور 'sudo sshd -T | grep -E 'permitrootlogin|passwordauthentication چه باید نشان دهد؟

پاسخ: permitrootlogin no و passwordauthentication no. اگر yes دیدی، یک فایل دیگر در sshd_config.d زودتر خوانده شده است.

▼ 2. بعد از bootstrap، چطور مطمئن می‌شوی جز 22، 80 و 443 چیزی از بیرون باز نیست؟

پاسخ: sudo ss -tlnp روی سرور (همه‌ی گوش‌دهنده‌های 0.0.0.0 را ببین)، و از بیرون nmap -Pn <ip>. ufw status کافی نیست چون پورت‌های Docker را نشان نمی‌دهد.

▼ 3. harden-ssh.ps1 را روی ویندوز اجرا کنی چه چیزی را باید عوض کنی؟

پاسخ: مسیر به C:\ProgramData\ssh\sshd_config، و sshd ویندوز sshd_config.d را پیش‌فرض Include نمی‌کند؛ باید خود sshd_config را ویرایش کنی و Restart-Service sshd.

جمله‌ی مصاحبه

A fresh server gets a sudo user with my key, SSH with keys only and no root, a firewall that allows 22, 80 and 443, fail2ban, automatic security updates and swap; and I prove the SSH change with sshd -T before I close the session that could still get me back in.

درس 3 · ops

A release: check, show the plan, ask, then ship

انتشار: اول چک، بعد نقشه، بعد تأیید، بعد ارسال

preflight checks | dry run | confirmation | git tag | rollback

بعد از این درس می‌توانی

- قبل از هر ارسال وضعیت همه‌ی repoها را چک کنی: شاخه‌ی درست، بدون تغییر commitنشده، عقب نبودن از GitHub.
- همه‌ی مشکل‌ها را یکجا گزارش کنی، با -DryRun فقط نقشه را نشان بدهی، و بدون «yes» صریح کاری نکنی.
- هر انتشار را با یک tag علامت بزنی تا بدانی دقیقاً چه چیزی روی سرور است و چطور برگردی.

سناریو از پروژه‌ی واقعی

release.ps1 در Control Panel با یک دستور همه‌چیز را از PC به سرور می‌فرستد، در هفت قدم که هر شکستی بقیه را متوقف می‌کند. قدم اول فقط نگاه می‌کند: پنج repo، شاخه، تغییرات، عقب یا جلو بودن، و حتی اینکه truth bank روی سرور commitی دارد که PC ندارد (قدم چهارم آن را بازنویسی می‌کرد). بعد نقشه را نشان می‌دهد و می‌پرسد. با -DryRun همان‌جا می‌ایستد. آخر کار هم release-tag-<تاریخ-ساعت> در هر repo. نمونه‌ی این درس همان قدم‌های چک، تأیید و tag است، در سه زبان، روی repoهای git واقعی با یک remote محلی.

سؤال چرا همه‌ی مشکل‌ها را جمع می‌کند و بعد متوقف می‌شود؟

پاسخ اگر اولین مشکل متوقف کند، درستش می‌کنی، دوباره اجرا می‌کنی و به دومی می‌رسی. با یک فهرست کامل، یک دور تعمیر کافی است.

سؤال tag وقتی commit hash هست چه فایده‌ای دارد؟

پاسخ release-20261022-2130 در هر پنج repo یعنی «این مجموعه با هم روی سرور بود». برگشت: git checkout آن tag در هر repo روی سرور و deploy.sh دوباره.

مفهوم‌ها

چک قبل از ارسال

fetch، بعد سه سؤال: آیا روی شاخه‌ای هستم که سرور می‌کشد (پیش‌فرض remote با ls-remote --symref)؟ تغییر commitنشده هست؟ remote چیزی دارد که من ندارم (rev-list --count HEAD..origin/main)؟ تعداد commitهای جلوتر هم برای نقشه چاپ می‌شود.

Dry run

همه‌ی چک‌ها و نقشه، بدون هیچ تغییر. اسکریپتی که dry run ندارد را اولین بار باید روی production امتحان کنی. قاعده: dry run باید از همان کد مسیر اصلی رد شود و درست قبل از اولین تغییر بایستد.

تأیید

فقط «yes» کامل قبول است؛ y، Enter یا ورودی خالی یعنی نه. برای CI یا خودکارسازی --yes- / Yes هست. در PowerShell، Read-Host زیر -NonInteractive خطا می‌دهد، پس نمونه از [Console]::In.ReadLine() می‌خواند که ورودی pipeشده را هم می‌گیرد.

ترتیب قدم‌ها

ارزان و بی‌خطر اول (چک، تست)، بعد چیزهایی که برگشت دارند (tag، push)، آخر چیزی که روی سرور اثر دارد (deploy، migrate) و در نهایت smoke test از بیرون: هر سایت همان کدی را بدهد که باید (۲۰۰، یا ۴۰۱ پشت basic auth).

مقایسه

پارامتر dry run	
Bash	`--dry-run (حلقه‌ی while/case)`
PowerShell	`[switch]$DryRun`
Python	`argparse: action='store_true'`

PowerShell و Python تجزیه‌ی آرگومان را مجانی می‌دهند؛ Bash دستی.

جمع کردن مشکل‌ها	
Bash	`problems+=("...")`
PowerShell	`$problems += '...'`
Python	`problems.append('...')`

+= در PowerShell هر بار آرایه‌ی تازه می‌سازد؛ برای ده تا مهم نیست، برای هزاران List[string].

پرسیدن از کاربر

Bash	`read -r -p '...' answer`
PowerShell	`[Console]::In.ReadLine()` (نه Read-Host)
Python	`input('...')` با `except EOFError`

بدون ترمینال، read شکست می‌خورد و input EOFError می‌دهد؛ هر دو باید «نه» حساب شوند.

خروجی git در متغیر

Bash	`x="$(git -C repo branch --show-current)"`
PowerShell	`$x = git -C repo branch --show-current`
Python	`run([...], capture_output=True, text=True).stdout.strip()`

PowerShell خروجی چندخطی را آرایه می‌کند؛ Bash و Python یک رشته.

نوشتن در stderr

Bash	`echo '...' >&2`
PowerShell	`[Console]::Error.WriteLine('...')`
Python	`print('...', file=sys.stderr)`

Write-Error در PowerShell با 'Stop' اسکریپت را همان‌جا متوقف می‌کند؛ برای فقط پیام، stderr مستقیم.

کد

چک همه‌ی repoها، همه‌ی مشکل‌ها یکجا

Bash کد اجراشده و تست‌شده در /examples examples/ops/release/release.sh

```
declare -A default=()
problems=()
for repo in "$@"; do
  name="$(basename "$repo")"
  git -C "$repo" fetch --quiet origin
  # The branch the remote calls default is the one the server pulls.
  default[$repo]="$(git -C "$repo" ls-remote --symref origin HEAD | sed -nE 's|^ref:
refs/heads/(\S+)\s+HEAD$|\1|p')"
  branch="$(git -C "$repo" branch --show-current)"
  behind="$(git -C "$repo" rev-list --count "HEAD..origin/${default[$repo]}")"
  ahead="$(git -C "$repo" rev-list --count "origin/${default[$repo]}..HEAD")"
  printf '  %-12s %-8s %s new commit(s)\n' "$name" "$branch" "$ahead"
  [[ "$branch" == "${default[$repo]}" ]] || problems+=("$name is on '$branch', not '${default[$repo]}'")
  [[ -z "$(git -C "$repo" status --porcelain --untracked-files=no)" ]] || problems+=("$name has uncommitted
changes")
  [[ "$behind" -eq 0 ]] || problems+=("$name is $behind commit(s) behind origin: pull first")
done
# Every problem of every repo at once, not just the first: one round of fixing.
if [[ ${#problems[@]} -gt 0 ]]; then
  printf '  x %s\n' "${problems[@]}" >&2
  exit 1
fi
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/release/release.ps1

```
$default = @{}
$problems = @()
foreach ($repo in $Repos) {
    $name = Split-Path $repo -Leaf
    Invoke-Git $repo fetch --quiet origin | Out-Null
    # The branch the remote calls default is the one the server pulls.
    $default[$repo] = (Invoke-Git $repo ls-remote --symref origin HEAD |
        Select-String '^ref: refs/heads/(\S+)\s+HEAD$').Matches[0].Groups[1].Value
    $branch = Invoke-Git $repo branch --show-current
    $behind = [int](Invoke-Git $repo rev-list --count "HEAD..origin/$($default[$repo])")
    $ahead = [int](Invoke-Git $repo rev-list --count "origin/$($default[$repo])..HEAD")
    '  {0,-12} {1,-8} {2} new commit(s)' -f $name, $branch, $ahead
    if ($branch -ne $default[$repo]) { $problems += "$name is on '$branch', not '$($default[$repo])'" }
    if (Invoke-Git $repo status --porcelain --untracked-files=no) { $problems += "$name has uncommitted
changes" }
    if ($behind -gt 0) { $problems += "$name is $behind commit(s) behind origin: pull first" }
}
# Every problem of every repo at once, not just the first: one round of fixing.
if ($problems) {
    $problems | ForEach-Object { [Console]::Error.WriteLine("  x $_") }
    exit 1
}
```

Python کد اجراشده و تست‌شده در /examples examples/ops/release/release.py

```
default: dict[str, str] = {}
problems: list[str] = []
for repo in args.repos:
    name = Path(repo).name
    git(repo, "fetch", "--quiet", "origin")
    # The branch the remote calls default is the one the server pulls.
    default[repo] = re.search(r"^ref: refs/heads/(\S+)\s+HEAD$", git(repo, "ls-remote", "--symref", "origin",
"HEAD"), re.M)[1]
    branch = git(repo, "branch", "--show-current")
    behind = int(git(repo, "rev-list", "--count", f"HEAD..origin/{default[repo]}"))
    ahead = int(git(repo, "rev-list", "--count", f"origin/{default[repo]}..HEAD"))
    print(f"  {name:<12} {branch:<8} {ahead} new commit(s)")
    if branch != default[repo]:
        problems.append(f"{name} is on '{branch}', not '{default[repo]}'")
    if git(repo, "status", "--porcelain", "--untracked-files=no"):
        problems.append(f"{name} has uncommitted changes")
    if behind:
        problems.append(f"{name} is {behind} commit(s) behind origin: pull first")
# Every problem of every repo at once, not just the first: one round of fixing.
if problems:
    print("\n".join(f"  x {p}" for p in problems), file=sys.stderr)
    sys.exit(1)
```

dry run، تأیید، push و tag

Bash کد اجراشده و تست‌شده در /examples examples/ops/release/release.sh

```
if $dry_run; then
  echo "Dry run: nothing changed. Would tag $tag."
  exit 0
fi
if ! $yes; then
  read -r -p "Push and tag $tag in $# repo(s)? (yes/no) " answer || answer=""
  [[ "$answer" == yes ]] || { echo "Stopped."; exit 1; }
fi
for repo in "$@"; do
  git -C "$repo" push --quiet origin "HEAD:${default[$repo]}"
  # The tag says exactly which commit is on the server, and is the way back.
  git -C "$repo" tag -a "$tag" -m "Released"
  git -C "$repo" push --quiet origin "$tag"
done
echo "Released $tag."
```

PowerShell کد اجراشده و تست‌شده در /examples `examples/ops/release/release.ps1`

```powershell
if ($DryRun) {
    "Dry run: nothing changed. Would tag $tag."
    exit 0
}
if (-not $Yes) {
    # Not Read-Host: it throws under -NonInteractive (CI, scheduled tasks), and this
    # reads a piped answer the same way as a typed one. The prompt goes to stderr, as
    # bash's read -p does, so it never mixes with the output.
    [Console]::Error.Write("Push and tag $tag in $($Repos.Count) repo(s)? (yes/no) ")
    if ([Console]::In.ReadLine() -ne 'yes') { 'Stopped.'; exit 1 }
}
foreach ($repo in $Repos) {
    Invoke-Git $repo push --quiet origin "HEAD:$($default[$repo])" | Out-Null
    # The tag says exactly which commit is on the server, and is the way back.
    Invoke-Git $repo tag -a $tag -m 'Released' | Out-Null
    Invoke-Git $repo push --quiet origin $tag | Out-Null
}
"Released $tag."
```

Python کد اجراشده و تست‌شده در /examples `examples/ops/release/release.py`

```python
if args.dry_run:
    print(f"Dry run: nothing changed. Would tag {tag}.")
    sys.exit(0)
if not args.yes:
    try:
        answer = input(f"Push and tag {tag} in {len(args.repos)} repo(s)? (yes/no) ")
    except EOFError:  # no terminal and nothing piped in
        answer = ""
    if answer != "yes":
        print("Stopped.")
        sys.exit(1)
for repo in args.repos:
    git(repo, "push", "--quiet", "origin", f"HEAD:{default[repo]}")
    # The tag says exactly which commit is on the server, and is the way back.
    git(repo, "tag", "-a", tag, "-m", "Released")
    git(repo, "push", "--quiet", "origin", tag)
print(f"Released {tag}.")
```

اشتباه‌های رایج

همه **deploy از شاخه‌ای که سرور نمی‌کشد**

چرا: push روی feature چیزی را روی سرور عوض نمی‌کند، ولی tag و smoke test می‌گویند «منتشر شد».

درست: شاخه‌ی پیش‌فرض را از remote بپرس و با شاخه‌ی فعلی مقایسه کن.

همه **ارسال وقتی سرور چیزی دارد که PC ندارد**

چرا: release.ps1 فایل‌های JobSearch را روی نسخه‌ی سرور می‌نویسد؛ ویرایش‌های داشبورد زنده بی‌صدا از بین می‌رفت.

درست: قبل از ارسال، تاریخچه‌ی سرور را بخوان و اگر commit اضافه دارد متوقف شو (-PullTruthBank).

PowerShell **Read-Host در اسکریپتی که از Task Scheduler یا CI اجرا می‌شود**

چرا: زیر -NonInteractive خطا می‌دهد؛ بدون آن منتظر ورودی‌ای می‌ماند که هیچ‌وقت نمی‌آید.

درست: سوییچ -Yes برای اجرای خودکار، و [Console]::In.ReadLine() برای حالت تعاملی.

همه **tag فقط در یک repo**

چرا: وقتی پنج repo با هم روی سرورند، نسخه‌ی یکی بدون بقیه چیزی را بازسازی نمی‌کند.

درست: یک نام tag برای همه‌ی repoهای یک انتشار.

این کد کجای پروژه‌هاست

`ControlPanel/deploy/vps/release.ps1:9`

هفت قدم به ترتیب؛ هر شکست بقیه را متوقف می‌کند

`ControlPanel/deploy/vps/release.ps1:95`

قدم ۱: شاخه، تغییرات، عقب بودن، و truth bank سرور

`ControlPanel/deploy/vps/release.ps1:156`

-DryRun و تأیید با yes

`ControlPanel/deploy/vps/release.ps1:178`

tag release-<stamp> در هر repo

`ControlPanel/deploy/vps/release.ps1:217`

smoke test: هر سایت کد مورد انتظارش، ۴۰۱ برای سایت‌های خصوصی

تمرین

▼ 1. در یک repo فایلی را تغییر بده و repo دیگر را روی شاخه‌ی دیگری ببر، بعد release.ps1 -DryRun. چند خط قرمز می‌بینی؟

پاسخ: دو خط، هر دو در همان اجرا: «has uncommitted changes» و «is on 'feature', not 'main'». تست‌های examples/ops همین را چک می‌کنند.

▼ 2. چرا قالب tag تا دقیقه است و نه ثانیه؟ اگر دو انتشار در یک دقیقه باشند چه می‌شود؟

پاسخ: خوانا است و دو انتشار در یک دقیقه نادر است. اگر پیش بیاید git tag خطا می‌دهد (tag already exists) و اسکریپت متوقف می‌شود، که درست است: هیچ چیز بی‌صدا بازنویسی نمی‌شود.

▼ 3. برگشت به انتشار قبلی روی سرور را بنویس.

پاسخ: در هر repo: git -C /srv/portfolio/<repo> checkout release-<قبلی>، بعد deploy.sh. توجه: deploy.sh با pull --ff-only روی HEAD جدا (detached) خطا می‌دهد؛ برای برگشت موقت یا pull را رد کن یا شاخه را reset کن.

جمله‌ی مصاحبه

A release script checks every repo first and lists every problem at once, prints the plan, stops there on a dry run, ships only on an explicit yes, and tags all repos with one release name so I always know what is on the server and how to go back.

درس 4 · ops

Deploying on the server: one at a time, fast-forward only, wait for health

استقرار روی سرور: یکی‌یکی، فقط fast-forward، صبر برای سلامت

idempotent deploy | health check | git pull --ff-only | file lock | flock

بعد از این درس می‌توانی

- با یک قفل فایل مطمئن شوی دو deploy هم‌زمان روی هم کار نمی‌کنند، و قفل بعد از crash گیر نمی‌کند.
- کد را فقط با fast-forward بکشی تا سرور هیچ‌وقت merge اختراع نکند.
- deploy را تا وقتی برنامه به health جواب نداده موفق حساب نکنی.

سناریو از پروژه‌ی واقعی

deploy.sh هم دستی اجرا می‌شود، هم از GitHub Actions بعد از هر push موفق، هم از قدم پنجم release.ps1. دو push پشت سر هم یعنی دو deploy که می‌توانند هم‌زمان git pull و docker compose up --build کنند. خط exec 9>/tmp/portfolio-deploy.lock; flock 9 دومی را منتظر نگه می‌دارد. بعد build، pull --ff-only، و حلقه‌ای که تا ۵ دقیقه منتظر می‌ماند همه‌ی سرویس‌ها healthy شوند؛ اگر نشدند کد ۱ و Action قرمز. نمونه‌ی این درس همان سه قدم است و تست‌ها هر سه را واقعاً امتحان می‌کنند: قفل گرفته‌شده، repo واگرا، و برنامه‌ای که جواب نمی‌دهد.

سؤال چرا قفل فایل و نه یک فایل «در حال اجرا» که آخر کار پاک شود؟

پاسخ اگر اسکریپت وسط کار بمیرد (kill، OOM، قطع SSH) فایل می‌ماند و deploy بعدی برای همیشه منتظر است. flock را kernel نگه می‌دارد و با بسته شدن فایل یا مرگ پروسه، هر طور که باشد، آزاد می‌کند.

سؤال چرا --ff-only؟ pull معمولی هم کار می‌کند.

پاسخ pull معمولی وقتی سرور commit خودش را داشته باشد یک merge commit می‌سازد که هیچ‌کس review نکرده. --ff-only در این حالت متوقف می‌شود و می‌گوید چیزی غیرعادی است.

مفهوم‌ها

قفل در سه زبان

Bash: exec 9>file باز می‌کند و flock 9 منتظر می‌ماند؛ flock -n فوراً شکست می‌خورد تا بشود پیام «منتظرم» داد. Python: fcntl.flock با همان قفل kernel، پس با Bash هم هماهنگ است. PowerShell: [IO.File]::Open(..., FileShare.None)؛ .NET روی لینوکس همان flock را می‌گیرد، ولی صبر نمی‌کند و استثنا می‌دهد، پس حلقه لازم است. روی ویندوز FileShare.None قفل واقعی ویندوز است.

idempotent

deploy.sh را صد بار پشت سر هم اجرا کنی نتیجه همان است: repo اگر نیست clone، اگر هست pull، و docker compose up فقط چیزی را که عوض شده دوباره می‌سازد. اسکریپتی که بار دوم خطا می‌دهد، در CI که retry می‌کند، دردسر می‌شود.

صبر برای health

اینکه docker compose up تمام شد یعنی کانتینرها شروع شدند، نه اینکه برنامه جواب می‌دهد. deploy.sh وضعیت HEALTHCHECK هر کانتینر را می‌خواند تا هیچ‌کدام starting نباشد، و اگر unhealthy ماند با کد ۱ تمام می‌شود. نمونه‌ی این درس همین را با یک URL نشان می‌دهد: هر ثانیه یک درخواست، تا سقف timeout.

Caddyfile و bind mount

یک نکته‌ی پنهان از deploy.sh: Caddyfile تک‌فایل mount شده. git pull فایل را با فایل تازه جایگزین می‌کند (inode جدید) و کانتینر هنوز نسخه‌ی قدیم را می‌بیند. deploy.sh قبل و بعد از pull hash می‌گیرد و اگر فرق کرد caddy را restart می‌کند.

مقایسه

گرفتن قفل	
Bash	`exec 9>lock; flock 9`
PowerShell	`[IO.File]::Open(lock, 'OpenOrCreate', 'ReadWrite', 'None')`
Python	`fcntl.flock(f, fcntl.LOCK_EX)`

PowerShell/.NET صبر نمی‌کند؛ IOException را بگیر و دوباره امتحان کن.

امتحان بدون صبر

Bash	`flock -n 9`
PowerShell	`(همیشه بدون صبر)`
Python	`LOCK_EX | LOCK_NB → BlockingIOError`

حالت بدون صبر برای چاپ «منتظرم» است، تا کسی فکر نکند اسکریپت هنگ کرده.

آزاد شدن قفل

Bash	`با پایان پروسه`
PowerShell	`$lock.Dispose() در finally`
Python	`با بسته شدن فایل یا پایان پروسه`

در هر سه، مرگ پروسه قفل را آزاد می‌کند؛ هیچ فایل قفل کهنه‌ای نمی‌ماند.

درخواست health

Bash	`curl -fsS -o /dev/null -m 5 "$url"`
PowerShell	`Invoke-WebRequest $url -TimeoutSec 5`
Python	`urllib.request.urlopen(url, timeout=5)`

curl بدون -f با ۵۰۰ هم موفق است؛ Invoke-WebRequest و urlopen روی ۴xx/۵xx استثنا می‌دهند.

حلقه با timeout

Bash	`for _ in $(seq "$timeout"); do ...; sleep 1; done`
PowerShell	`foreach ($i in 1..$timeout) { ... }`
Python	`for _ in range(timeout): ...`

بعد از حلقه یعنی شکست: هر سه با پیام و کد غیر صفر تمام می‌شوند.

کد

یک deploy در هر لحظه

Bash کد اجراشده و تست‌شده در examples/ examples/ops/deploy/deploy.sh

```bash
# One deploy at a time: a push that lands during a deploy waits for it instead of
# pulling and rebuilding over it. The kernel drops the lock when this process ends,
# however it ends, so a crashed deploy never leaves a stale lock behind.
exec 9>"$lock"
if ! flock -n 9; then
  echo "another deploy is running; waiting"
  flock 9
fi
```

PowerShell کد اجراشده و تست‌شده در examples/ examples/ops/deploy/deploy.ps1

```powershell
# One deploy at a time. FileShare.None is an exclusive lock (on Linux .NET takes the
# same flock as bash's `flock`); it fails at once instead of waiting, hence the loop.
# The lock goes when the handle is closed or the process ends.
$lock = $null
$said = $false
while (-not $lock) {
    try { $lock = [IO.File]::Open($lockFile, 'OpenOrCreate', 'ReadWrite', 'None') }
    catch [IO.IOException] {
        if (-not $said) { 'another deploy is running; waiting'; $said = $true }
        Start-Sleep -Milliseconds 500
    }
}
```

Python کد اجراشده و تست‌شده در examples/ examples/ops/deploy/deploy.py

```python
# One deploy at a time: the same flock(2) as bash's `flock`, so the two even exclude
# each other. Released when the file is closed or the process ends, however it ends.
lock = open(lock_file, "w")  # not `with`: it is held for the whole run
try:
    fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
except BlockingIOError:
    print("another deploy is running; waiting", flush=True)
    fcntl.flock(lock, fcntl.LOCK_EX)
```

pull فقط با fast-forward

Bash کد اجراشده و تست‌شده در examples/ examples/ops/deploy/deploy.sh

```bash
for dir in "$root"/*/; do
  [[ -d "$dir/.git" ]] || continue
  echo "pull $(basename "$dir")"
  # --ff-only: the server's copy never has commits of its own. If it somehow does,
  # stop here rather than invent a merge commit nobody reviewed.
  git -C "$dir" pull --quiet --ff-only
done
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/deploy/deploy.ps1

```powershell
foreach ($dir in Get-ChildItem -LiteralPath $Root -Directory) {
    if (-not (Test-Path (Join-Path $dir.FullName '.git'))) { continue }
    "pull $($dir.Name)"
    # --ff-only: the server's copy never has commits of its own. If it somehow
    # does, stop here rather than invent a merge commit nobody reviewed.
    git -C $dir.FullName pull --quiet --ff-only
    if ($LASTEXITCODE -ne 0) { throw "git pull --ff-only failed in $($dir.Name)" }
}
```

Python کد اجراشده و تست‌شده در /examples examples/ops/deploy/deploy.py

```python
for d in sorted(root.iterdir()):
    if not (d / ".git").is_dir():
        continue
    print(f"pull {d.name}", flush=True)
    # --ff-only: the server's copy never has commits of its own. If it somehow does,
    # stop here rather than invent a merge commit nobody reviewed.
    subprocess.run(["git", "-C", str(d), "pull", "--quiet", "--ff-only"], check=True)
```

موفق فقط وقتی برنامه جواب داد

Bash کد اجراشده و تست‌شده در /examples examples/ops/deploy/deploy.sh

```bash
for _ in $(seq "$timeout"); do
  if curl -fsS -o /dev/null -m 5 "$url"; then
    echo "deploy ok"
    exit 0
  fi
  sleep 1
done
echo "not healthy after ${timeout}s: $url" >&2
exit 1
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/deploy/deploy.ps1

```powershell
foreach ($i in 1..$timeout) {
    try {
        $r = Invoke-WebRequest -Uri $url -TimeoutSec 5
        if ($r.StatusCode -eq 200) { 'deploy ok'; exit 0 }
    }
    catch { Start-Sleep -Seconds 1 }
}
throw "not healthy after ${timeout}s: $url"
```

Python کد اجراشده و تست‌شده در /examples examples/ops/deploy/deploy.py

```python
for _ in range(timeout):
    try:
        with urllib.request.urlopen(url, timeout=5) as r:
            if r.status == 200:
                print("deploy ok")
                sys.exit(0)
    except OSError:  # refused, timed out, or an HTTP error status (HTTPError is an OSError)
        pass
    time.sleep(1)
sys.exit(f"not healthy after {timeout}s: {url}")
```

اشتباه‌های رایج

همه **فایل pid یا «در حال اجرا» به‌جای قفل**

چرا: بعد از یک crash فایل می‌ماند و همه‌ی deployهای بعدی گیر می‌کنند، یا کسی آن را دستی پاک می‌کند وقتی واقعاً deployی در حال اجراست.

درست: flock (یا FileShare.None)؛ kernel صاحب قفل است.

همه **deploy موفق = docker compose up تمام شد**

چرا: کانتینر بالا آمده ولی برنامه هنوز migration می‌کند یا crash-loop است. Action سبز است و سایت ۵۰۲ می‌دهد.

درست: تا healthy شدن صبر کن و در غیر این صورت کد ۱.

همه **docker compose up --wait وقتی سرویس‌های یک‌باره (db-init، seed) هست**

چرا: بعضی نسخه‌های Compose خروج عادی آن‌ها را شکست حساب می‌کنند.

درست: حلقه‌ی خودت روی 'docker compose ps --format {{.Health}}'، مثل deploy.sh.

Bash **curl بدون -f در health check**

چرا: curl روی ۵۰۰ هم کد ۰ برمی‌گرداند؛ health check همیشه موفق است.

درست: curl -fsS.

PowerShell **catch بدون نوع دور [IO.File]::Open**

چرا: مسیر اشتباه یا نبود دسترسی هم IOException نیست و باید فوراً خطا بدهد، نه اینکه برای همیشه «منتظر» بماند.

درست: catch [IO.IOException]؛ بقیه‌ی خطاها بالا بروند.

این کد کجای پروژه‌هاست

ControlPanel/deploy/vps/deploy.sh:28

exec 9>/tmp/portfolio-deploy.lock; flock 9

`ControlPanel/deploy/vps/deploy.sh:41`

git pull --ff-only برای هر repo

`ControlPanel/deploy/vps/deploy.sh:57`

Caddyfile عوض شد؟ caddy را restart کن

`ControlPanel/deploy/vps/deploy.sh:62`

صبر برای health، نه up --wait، و کد ۱ اگر سالم نشد

تمرین

▼ 1. روی سرور، در یک ترمینال flock /tmp/portfolio-deploy.lock sleep 60 و در دیگری deploy.sh. چه می‌بینی؟

پاسخ: deploy.sh واقعی پیامی نمی‌دهد و فقط صبر می‌کند (flock بدون -n). نسخه‌ی این درس اول می‌گوید «another deploy is running; waiting». بعد از ۶۰ ثانیه هر دو ادامه می‌دهند.

▼ 2. روی سرور یک commit در ControlPanel بساز و deploy.sh را اجرا کن. چرا شکست می‌خورد و چطور درستش می‌کنی؟

پاسخ: اگر GitHub هم commit تازه دارد، تاریخچه‌ها واگرا شده‌اند و --ff-only رد می‌کند. commit سرور را به PC ببر (اگر لازم است)، بعد روی سرور git reset --hard origin/main.

▼ 3. deploy.ps1 روی ویندوز و deploy.sh روی همان ماشین در WSL هم‌زمان اجرا شوند؛ قفل جلویشان را می‌گیرد؟

پاسخ: نه. قفل ویندوز (FileShare.None) و flock لینوکس دو سیستم جدا هستند، حتی روی یک فایل در /mnt/c. قفل فقط بین پروسه‌هایی کار می‌کند که از یک نوع قفل استفاده کنند.

جمله‌ی مصاحبه

A deploy takes a kernel file lock so only one runs at a time and a crash can't leave it stuck, pulls with --ff-only so the server never invents a merge, and only reports success once the app answers its health check.

درس 5 · ops

The nightly job: backup, retention and a log

کار شبانه: بکاپ، نگه‌داری ۱۴ روزه و لاگ

cron | systemd timer | Task Scheduler | retention | consistent backup | checksum

بعد از این درس می‌توانی

- از یک دیتابیس در حال کار بکاپ سازگار بگیری، فشرده کنی و checksum کنارش بگذاری.
- بکاپ‌های قدیمی‌تر از N روز را پاک کنی و بدانی find -mtime دقیقاً چه چیزی را می‌شمارد.
- کار را هر شب زمان‌بندی کنی (cron، systemd timer یا Task Scheduler) و خروجی‌اش را جایی بنویسی که بشود خواند.

سناریو از پروژه‌ی واقعی

backup.sh روی VPS ساعت ۳:۳۰ هر شب از cron اجرا می‌شود: چهار دیتابیس SQL Server با BACKUP DATABASE ... WITH COPY_ONLY, CHECKSUM، MongoDB ریپایومد با mongodump، و سه volume فایل‌های آپلودی. نتیجه در /srv/backups/daily/<stamp>، و هر چیز قدیمی‌تر از ۱۴ روز پاک می‌شود. آخرین اجرا امشب ۳:۳۰ بود و همه‌ی بخش‌ها ok شدند. نمونه‌ی این درس همان شکل را با SQLite نشان می‌دهد (چون در کانتینر تست سبک است)، و تست‌ها یک تفاوت پنهان را ثابت می‌کنند: پوشه‌ی ۱۴.۵ روزه را find نگه می‌دارد و PowerShell و Python پاک می‌کنند.

سؤال چرا cp فایل دیتابیس کافی نیست؟

پاسخ اگر برنامه وسط کپی بنویسد، نیمی از فایل قبل از تغییر است و نیمی بعد از آن. ابزار بکاپ خود دیتابیس (sqlite، mongodump، BACKUP DATABASE، backup.) یک عکس سازگار می‌گیرد.

سؤال COPY_ONLY در backup.sh برای چیست؟

پاسخ بکاپ کامل معمولی زنجیره‌ی بکاپ‌های differential را از نو شروع می‌کند. COPY_ONLY یک بکاپ مستقل است که به برنامه‌ی بکاپ دیگری دست نمی‌زند.

مفهوم‌ها

cron

30 3 * * * یعنی دقیقه ۳۰، ساعت ۳، هر روز. cron با PATH کوتاه (/usr/bin:/bin) و بدون profile اجرا می‌کند؛ مسیرها را کامل بنویس. >> log 2>&1 هم خروجی و هم خطا را به لاگ می‌فرستد (ترتیبش مهم است: 2>&1 >> log خطا را جا می‌اندازد). اگر سرور ساعت ۳:۳۰ خاموش بود، cron آن شب را جبران نمی‌کند.

systemd timer

دو فایل: backup.service (ExecStart=/srv/.../backup.sh) و backup.timer (OnCalendar=*-*-* 03:30، Persistent=true). Persistent یعنی اجرای جاافتاده بعد از روشن شدن انجام شود. لاگ خودکار در journalctl -u backup، کد خروج در systemctl status، و زمان اجرای بعدی را نشان می‌دهد.

Task Scheduler ویندوز

Register-ScheduledTask با New-ScheduledTaskTrigger -Daily -At 3:30am و یک Action که pwsh -NoProfile -NonInteractive -File را اجرا می‌کند. «Run whether user is logged on or not» و «Run task as soon as possible after a scheduled start is missed» معادل Persistent هستند. کد خروج در Last Run Result.

نگه‌داری و checksum

find -mtime +14 سن را به روز کامل گرد پایین می‌کند و فقط ۱۵ روز و بیشتر را پاک می‌کند؛ PowerShell و Python دقیقاً ۲۴×۱۴ ساعت را مقایسه می‌کنند. SHA256SUMS کنار هر بکاپ به فرمت sha256sum است تا هر کپی در هر جا با sha256sum -c قابل اثبات باشد.

مقایسه

زمان‌بندی	
Bash	`crontab: 30 3 * * * /path/backup.sh >> log 2>&1`
PowerShell	`Register-ScheduledTask -Trigger (New-ScheduledTaskTrigger -Daily -At 3:30am)`
Python	`systemd timer یا cron زمان‌بند ندارد؛ از`

cron اجرای جاافتاده را جبران نمی‌کند؛ systemd با Persistent=true و Task Scheduler با گزینه‌اش جبران می‌کنند.

بکاپ سازگار SQLite	
Bash	`sqlite3 app.db ".backup 'out/app.db'"`
PowerShell	`sqlite3 $db ".backup '$out/app.db'" + $LASTEXITCODE`
Python	`src.backup(dst)`

Python API بکاپ را در ماژول استاندارد دارد؛ بقیه CLI را صدا می‌زنند.

gzip

Bash	`gzip file`
PowerShell	`[IO.Compression.GZipStream]`
Python	`gzip.open(..., 'wb')`

Compress-Archive پاورشل zip می‌سازد، نه gz.

checksum

Bash	`sha256sum app.db.gz > SHA256SUMS`
PowerShell	`(Get-FileHash -Algorithm SHA256).Hash.ToLower()`
Python	`hashlib.sha256(data).hexdigest()`

Get-FileHash حروف بزرگ می‌دهد؛ برای sha256sum -c کوچکش کن.

پاک کردن قدیمی‌تر از ۱۴ روز

Bash	`find daily -mindepth 1 -maxdepth 1 -type d -mtime +14 -exec rm -rf {} +`
PowerShell	`Get-ChildItem -Directory | Where-Object LastWriteTime -lt (Get-Date).AddDays(-14) | Remove-Item -Recurse`
Python	`if d.stat().st_mtime < time.time() - 14 * 86400: shutil.rmtree(d)`

find روز کامل می‌شمارد: پوشه‌ی ۱۴.۵ روزه می‌ماند. دو تای دیگر آن را پاک می‌کنند. تست ثابتش می‌کند.

کد

بکاپ سازگار، فشرده، با checksum

Bash کد اجراشده و تست‌شده در /examples examples/ops/backup/backup.sh

```
db="${DB:?set DB to the database file}"
backups="${BACKUP_DIR:-/srv/backups}"
keep="${KEEP_DAYS:-14}"
out="$backups/daily/$(date +%Y%m%d-%H%M%S)"

mkdir -p "$out"
echo "[$(date -Is)] backup -> $out"
# .backup takes a consistent copy while the app keeps writing; cp of a live
# database file can catch it halfway through a write.
sqlite3 "$db" ".backup '$out/app.db'"
gzip "$out/app.db"
# The checksum travels with the backup: a copy elsewhere can prove it is intact.
(cd "$out" && sha256sum app.db.gz > SHA256SUMS)
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/backup/backup.ps1

```
$db = $env:DB
if (-not $db) { throw 'set DB to the database file' }
$backups = if ($env:BACKUP_DIR) { $env:BACKUP_DIR } else { '/srv/backups' }
$keep = if ($env:KEEP_DAYS) { [int]$env:KEEP_DAYS } else { 14 }
$out = Join-Path $backups "daily/$(Get-Date -Format 'yyyyMMdd-HHmmss')"

New-Item -ItemType Directory -Force $out | Out-Null
"[$(Get-Date -Format o)] backup -> $out"
# .backup takes a consistent copy while the app keeps writing; Copy-Item of a live
# database file can catch it halfway through a write.
sqlite3 $db ".backup '$out/app.db'"
if ($LASTEXITCODE -ne 0) { throw "sqlite3 .backup failed ($LASTEXITCODE)" }

$source = [IO.File]::OpenRead("$out/app.db")
$target = [IO.Compression.GZipStream]::new([IO.File]::Create("$out/app.db.gz"),
[IO.Compression.CompressionLevel]::Optimal)
try { $source.CopyTo($target) } finally { $target.Dispose(); $source.Dispose() }
Remove-Item "$out/app.db"
# The checksum travels with the backup, in sha256sum's format so `sha256sum -c` reads it.
$hash = (Get-FileHash "$out/app.db.gz" -Algorithm SHA256).Hash.ToLower()
Set-Content "$out/SHA256SUMS" "$hash  app.db.gz"
```

Python کد اجراشده و تست‌شده در /examples examples/ops/backup/backup.py

```
db = os.environ["DB"]
backups = Path(os.environ.get("BACKUP_DIR", "/srv/backups"))
keep = int(os.environ.get("KEEP_DAYS", "14"))
out = backups / "daily" / f"{datetime.now():%Y%m%d-%H%M%S}"

out.mkdir(parents=True)
print(f"[{datetime.now().isoformat(timespec='seconds')}] backup -> {out}", flush=True)
# The backup API takes a consistent copy while the app keeps writing; shutil.copy of a
# live database file can catch it halfway through a write. closing(): `with` on a
# bare sqlite3 connection only commits, it does not close the file.
with closing(sqlite3.connect(db)) as src, closing(sqlite3.connect(out / "app.db")) as dst:
    src.backup(dst)
with open(out / "app.db", "rb") as f, gzip.open(out / "app.db.gz", "wb") as z:
    shutil.copyfileobj(f, z)
(out / "app.db").unlink()
# The checksum travels with the backup, in sha256sum's format so `sha256sum -c` reads it.
digest = hashlib.sha256((out / "app.db.gz").read_bytes()).hexdigest()
(out / "SHA256SUMS").write_text(f"{digest}  app.db.gz\n")
```

نگه‌داری ۱۴ روز

Bash کد اجراشده و تست‌شده در /examples examples/ops/backup/backup.sh

```
# -mtime +14 means "15 days or more": find counts whole days and drops the fraction.
find "$backups/daily" -mindepth 1 -maxdepth 1 -type d -mtime +"$keep" -print -exec rm -rf {} +
echo "[$(date -Is)] done ($(du -sh "$out" | cut -f1))"
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/backup/backup.ps1

```
# Exactly KEEP_DAYS x 24 hours, unlike find -mtime, which rounds down to whole days.
$cutoff = (Get-Date).AddDays(-$keep)
Get-ChildItem (Join-Path $backups 'daily') -Directory |
    Where-Object LastWriteTime -lt $cutoff |
    ForEach-Object { $_.FullName; Remove-Item $_.FullName -Recurse -Force }
"[$(Get-Date -Format o)] done ($([math]::Round((Get-Item "$out/app.db.gz").Length / 1KB)) KB)"
```

Python کد اجراشده و تست‌شده در /examples examples/ops/backup/backup.py

```
# Exactly KEEP_DAYS x 24 hours, unlike find -mtime, which rounds down to whole days.
cutoff = time.time() - keep * 86400
for d in (backups / "daily").iterdir():
    if d.is_dir() and d.stat().st_mtime < cutoff:
        print(d)
        shutil.rmtree(d)
print(f"[{datetime.now().isoformat(timespec='seconds')}] done ({(out / 'app.db.gz').stat().st_size // 1024}
KB)")
```

همان کار برای SQL Server در کانتینر (backup.sh روی VPS)

همه‌ی زبان‌ها از کد واقعی پروژه (کوتاه‌شده)

```
for db in "${DATABASES[@]}"; do
  # Express can't compress backups; gzip does it afterwards.
  sql "BACKUP DATABASE [$db] TO DISK = N'/var/opt/mssql/backup/$db.bak' WITH INIT, COPY_ONLY, CHECKSUM"
>/dev/null
  # Read it through the container: the file belongs to mssql (uid 10001).
  docker compose exec -T sqlserver cat "/var/opt/mssql/backup/$db.bak" | gzip > "$OUT/$db.bak.gz"
  docker compose exec -T sqlserver rm -f "/var/opt/mssql/backup/$db.bak"
  echo "  $db ok"
done

docker compose exec -T rebiomed-mongo mongodump --quiet --archive --gzip > "$OUT/rebiomed-mongo.archive.gz"
```

اشتباه‌های رایج

Bash **cron با دستوری که در ترمینال کار می‌کند**

چرا: cron PATH و متغیرهای profile را ندارد؛ docker یا sqlcmd پیدا نمی‌شود و خطا فقط در ایمیلی می‌رود که کسی نمی‌خواند.

درست: مسیر کامل اسکریپت، cd داخل اسکریپت، و >> log 2>&1.

همه **لاگی که فقط «شروع» دارد**

چرا: اگر اسکریپت وسط کار بمیرد، لاگ نمی‌گوید کجا. بدون خط done نمی‌دانی موفق بود.

درست: زمان شروع، یک خط ok برای هر بخش، و زمان پایان با حجم؛ مثل backup.sh.

Python **with sqlite3.connect(...) as conn برای بستن اتصال**

چرا: context manager اتصال sqlite3 فقط commit یا rollback می‌کند و فایل را باز نگه می‌دارد.

درست: contextlib.closing(sqlite3.connect(...)).

PowerShell **Compress-Archive برای فایل .gz**

چرا: zip می‌سازد؛ gunzip یا gzip -t رویش شکست می‌خورد.

درست: GZipStream، یا اگر zip می‌خواهی پسوند .zip.

همه **پاک کردن قدیمی‌ها قبل از ساختن بکاپ امشب**

چرا: اگر بکاپ امشب شکست بخورد، یک بکاپ خوب کمتر داری.

درست: اول بکاپ تازه، بعد پاک کردن؛ با set -e شکست بکاپ به پاک کردن نمی‌رسد.

این کد کجای پروژه‌هاست

`ControlPanel/deploy/vps/backup.sh:8`

«این در برابر اشتباه محافظت می‌کند، نه در برابر از دست دادن سرور»

`ControlPanel/deploy/vps/backup.sh:34`

BACKUP DATABASE ... WITH INIT, COPY_ONLY, CHECKSUM

`ControlPanel/deploy/vps/backup.sh:52`

نگه‌داری ۱۴ روزه با find -mtime

`ControlPanel/deploy/vps/README.md:163`

خط crontab ساعت ۳:۳۰ با لاگ

تمرین

▼ 1. backup.sh را به یک systemd timer منتقل کن. دو فایل را بنویس.

پاسخ: /etc/systemd/system/portfolio-backup.service: [Service] Type=oneshot, User=deploy, ExecStart=/srv/portfolio/ControlPanel/deploy/vps/backup.sh. و portfolio-backup.timer: [Timer] OnCalendar=*-*-* 03:30, Persistent=true; [Install] WantedBy=timers.target. بعد systemctl enable --now portfolio-backup.timer.

▼ 2. پوشه‌ای که دقیقاً ۱۴ روز و ۱۲ ساعت پیش ساخته شده با KEEP_DAYS=14 در هر زبان چه می‌شود؟

پاسخ: Bash (find -mtime +14) نگهش می‌دارد چون ۱۴.۵ به ۱۴ گرد می‌شود و ۱۴ > ۱۴ نیست. PowerShell و Python پاکش می‌کنند. تست‌های examples/ops هر سه را چک می‌کنند.

▼ 3. KEEP_DAYS روی VPS ۱۴ است، ولی در /srv/backups/daily پانزده پوشه هست. اشکال است؟

پاسخ: نه، همان گرد کردن find است: پوشه‌ی دقیقاً ۱۴ روز و چند ساعته هنوز «۱۴ روز» حساب می‌شود و -mtime +14 آن را نگه می‌دارد. با اجرای ۳:۳۰ هر شب، همیشه ۱۵ پوشه می‌ماند.

▼ 4. از کجا می‌فهمی بکاپ دیشب اجرا شده؟

پاسخ: tail /srv/backups/backup.log: خط done با تاریخ امروز. بدون آن خط یعنی یا اجرا نشده یا وسط کار مرده. بهتر: یک health check بیرونی (مثل healthchecks.io) که اسکریپت در پایان صدایش کند و اگر نکرد به تو خبر دهد.

جمله‌ی مصاحبه

The nightly job takes a consistent backup with the database's own tool, compresses it, writes a checksum next to it, deletes old ones only after the new one exists, and logs start, every part and the end, so the log alone tells me whether last night worked.

درس 6 · ops

A backup that was never restored is not a backup

بازگردانی: بکاپی که restore نشده، بکاپ نیست

restore test | off-site copy | 3-2-1 | rclone | restic | integrity check

بعد از این درس می‌توانی

- یک کپی شبانه از بکاپ را بیرون از سرور بفرستی، در حساب و ارائه‌دهنده‌ی دیگری.
- هر شب خودکار ثابت کنی همان کپی بیرونی restore می‌شود: integrity check، checksum و داده‌ی واقعی.
- بدانی restore یک دیتابیس SQL Server روی سرور تازه چه چیزهایی جز خود فایل لازم دارد.

سناریو از پروژه‌ی واقعی

بکاپ شبانه‌ی VPS درست کار می‌کند، ولی بررسی سرور دو چیز نشان داد. اول: هر ۱۵ بکاپ فقط روی خود سرور است. هیچ ابزار کپی بیرونی (rclone یا restic) نصب نیست، و README خود پروژه هم می‌گوید «این بکاپ‌ها با سرور می‌میرند». دوم: از لاگ‌ها معلوم نیست restore هیچ‌وقت آزمایش شده باشد. این درس هر دو را درست می‌کند: restore-test بعد از بکاپ، جدیدترین پوشه را به مقصد بیرونی کپی می‌کند و بعد از روی همان کپی، نه نسخه‌ی اصلی، restore می‌کند. تست‌ها سه حالت خراب را هم امتحان می‌کنند: فایل آسیب‌دیده، دیتابیس خالی و نبود بکاپ.

سؤال بکاپ روی همان سرور در برابر چه چیزی محافظت نمی‌کند؟

پاسخ در برابر هر چیزی که خود سرور را ببرد: خرابی دیسک، حذف اشتباه VPS، قطع حساب ارائه‌دهنده، یا نفوذی که rm -rf /srv می‌کند. بکاپ و اصل یک‌جا می‌میرند.

سؤال چرا restore را از کپی بیرونی تست کنیم و نه از اصل؟

پاسخ روزی که سرور از دست رفت، فقط کپی بیرونی داری. اگر کپی ناقص منتقل شده یا مقصد خرابش کرده، باید امشب بفهمی، نه آن روز.

مفهوم‌ها

قاعده‌ی ۳-۲-۱

سه نسخه از داده، روی دو نوع رسانه‌ی مختلف، یک نسخه بیرون از محل. برای این VPS: داده‌ی زنده، بکاپ‌های /srv/backups، و یک کپی در ارائه‌دهنده‌ی دیگر (Backblaze B2، یک bucket در S3، یا حتی PC خانه). snapshot خود ارائه‌دهنده هم کمک می‌کند ولی در همان حساب است.

rclone یا restic

rclone copy /srv/backups/daily remote:portfolio فایل‌ها را همان‌طور که هست به ده‌ها نوع مقصد می‌فرستد. restic بکاپ رمزنگاری‌شده و deduplicate می‌سازد: هر snapshot فقط تکه‌های تازه را می‌فرستد و ارائه‌دهنده محتوا را نمی‌بیند. برای داده‌ای مثل جست‌وجوی شغلی و اطلاعات تماس، رمزنگاری سمت سرور ضروری است. کلید دسترسی مقصد فقط باید اجازه‌ی نوشتن داشته باشد، نه پاک کردن، تا نفوذ به سرور به بکاپ‌ها نرسد.

تست restore چه چیزی را ثابت می‌کند

سه لایه: checksum (فایل همان است که ساخته شد)، باز کردن و integrity check (فایل یک دیتابیس سالم است)، و یک پرس‌وجوی واقعی با حداقل تعداد ردیف (دیتابیس خالی هم کاملاً restore می‌شود). در SQL Server معادلش RESTORE VERIFYONLY و بعد RESTORE به نام موقت و DBCC CHECKDB است.

restore روی سرور تازه

restore-db.sh نشان می‌دهد فایل .bak کافی نیست: مسیرهای ویندوزی داخل بکاپ با MOVE به /var/opt/mssql/data/ می‌روند، و loginها در master هستند نه در دیتابیس، پس کاربرهای دیتابیس یتیم‌اند و باید با رمزهای .env دوباره ساخته شوند. یعنی .env هم بخشی از بکاپ است و باید جای امنی بیرون از سرور باشد (password manager).

مقایسه

جدیدترین بکاپ

Bash	`find daily -mindepth 1 -maxdepth 1 -type d | sort | tail -n 1`
PowerShell	`Get-ChildItem -Directory | Sort-Object Name | Select-Object -Last 1`
Python	`sorted(d for d in daily.iterdir() if d.is_dir())[-1]`

نام پوشه‌ها yyyyMMdd-HHmm است تا ترتیب الفبایی همان ترتیب زمانی باشد.

کپی پوشه

Bash	`[[ -d dst ]] || cp -r src dst`
PowerShell	`if (-not (Test-Path dst)) { Copy-Item src dst -Recurse }`
Python	`shutil.copytree(src, dst)`

cp -r و Copy-Item -Recurse به پوشه‌ای که هست، src را داخلش می‌گذارند (dst/src).

چک checksum

Bash	`sha256sum --quiet -c SHA256SUMS`
PowerShell	`(Get-FileHash ...).Hash -ne $expected`
Python	`hashlib.sha256(...).hexdigest() != expected`

-ne در PowerShell حروف بزرگ و کوچک را یکی می‌گیرد؛ != در Python نه، پس هر دو را hex کوچک نگه دار.

پوشه‌ی موقت که پاک شود

Bash	`tmp="$(mktemp -d)"; trap 'rm -rf "$tmp"' EXIT`
PowerShell	`try { ... } finally { Remove-Item $tmp -Recurse -Force }`
Python	`with tempfile.TemporaryDirectory() as tmp:`

trap EXIT در Bash روی exit و set -e هم اجرا می‌شود.

اثبات سلامت دیتابیس

Bash	`sqlite3 db 'PRAGMA integrity_check;'`
PowerShell	`sqlite3 $db 'PRAGMA integrity_check;' + $LASTEXITCODE`
Python	`conn.execute('PRAGMA integrity_check').fetchone()[0]`

SQL Server: DBCC CHECKDB. MongoDB: mongorestore به یک دیتابیس موقت و شمردن سندها.

کد

کپی بیرون از سرور

Bash کد اجراشده و تست‌شده در examples/ examples/ops/restore/restore-test.sh

```bash
latest="$(find "$backups/daily" -mindepth 1 -maxdepth 1 -type d | sort | tail -n 1)"
[[ -n "$latest" ]] || { echo "no backup in $backups/daily" >&2; exit 1; }
stamp="$(basename "$latest")"
copy="$offsite/$stamp"
# On the VPS this is `rclone copy "$latest" offsite:portfolio/$stamp` (or restic):
# another provider and another account, so one mistake or one breach can't reach both.
# Here a folder stands in for that remote. cp -r into an existing folder would nest it.
mkdir -p "$offsite"
[[ -d "$copy" ]] || cp -r "$latest" "$copy"
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/restore/restore-test.ps1

```powershell
$latest = Get-ChildItem (Join-Path $backups 'daily') -Directory | Sort-Object Name | Select-Object -Last 1
if (-not $latest) { throw "no backup in $backups/daily" }
$copy = Join-Path $offsite $latest.Name
# On the VPS this is `rclone copy` (or restic) to another provider and another account.
# Here a folder stands in for that remote. Copy-Item -Recurse into an existing folder
# would nest it, so only copy what is not there yet.
New-Item -ItemType Directory -Force $offsite | Out-Null
if (-not (Test-Path $copy)) { Copy-Item $latest.FullName $copy -Recurse }
```

Python کد اجراشده و تست‌شده در /examples examples/ops/restore/restore_test.py

```python
daily = sorted(d for d in (backups / "daily").iterdir() if d.is_dir())
if not daily:
    sys.exit(f"no backup in {backups / 'daily'}")
latest = daily[-1]
copy = offsite / latest.name
# On the VPS this is `rclone copy` (or restic) to another provider and another account.
# Here a folder stands in for that remote. Only what is not there yet is copied.
offsite.mkdir(parents=True, exist_ok=True)
if not copy.exists():
    shutil.copytree(latest, copy)
```

restore از همان کپی، و اثباتش

Bash کد اجراشده و تست‌شده در /examples examples/ops/restore/restore-test.sh

```bash
# From the copy you would have after losing the server, not from the original.
(cd "$copy" && sha256sum --quiet -c SHA256SUMS) || { echo "checksum mismatch in $copy" >&2; exit 1; }
tmp="$(mktemp -d)"
trap 'rm -rf "$tmp"' EXIT
gunzip -c "$copy/app.db.gz" > "$tmp/restored.db"
check="$(sqlite3 "$tmp/restored.db" 'PRAGMA integrity_check;')"
[[ "$check" == ok ]] || { echo "integrity check: $check" >&2; exit 1; }
# A backup of an empty database restores perfectly; it is still no backup.
rows="$(sqlite3 "$tmp/restored.db" "SELECT count(*) FROM $table;")"
[[ "$rows" -ge "$min" ]] || { echo "only $rows row(s) in $table, expected $min or more" >&2; exit 1; }
echo "restore test ok: $stamp, $rows rows in $table"
```

PowerShell کد اجراشده و تست‌شده در /examples examples/ops/restore/restore-test.ps1

```powershell
# From the copy you would have after losing the server, not from the original.
$expected = ((Get-Content (Join-Path $copy 'SHA256SUMS')) -split '\s+')[0]
$actual = (Get-FileHash (Join-Path $copy 'app.db.gz') -Algorithm SHA256).Hash
if ($actual -ne $expected) { throw "checksum mismatch in $copy" }   # -ne ignores case

$tmp = Join-Path ([IO.Path]::GetTempPath()) ([guid]::NewGuid())
New-Item -ItemType Directory $tmp | Out-Null
try {
    $restored = Join-Path $tmp 'restored.db'
    $zip = [IO.Compression.GZipStream]::new([IO.File]::OpenRead((Join-Path $copy 'app.db.gz')),
[IO.Compression.CompressionMode]::Decompress)
    $file = [IO.File]::Create($restored)
    try { $zip.CopyTo($file) } finally { $file.Dispose(); $zip.Dispose() }

    $check = sqlite3 $restored 'PRAGMA integrity_check;'
    if ($LASTEXITCODE -ne 0 -or $check -ne 'ok') { throw "integrity check: $check" }
    # A backup of an empty database restores perfectly; it is still no backup.
    $rows = [int](sqlite3 $restored "SELECT count(*) FROM $table;")
    if ($LASTEXITCODE -ne 0) { throw "no table $table in the backup" }
    if ($rows -lt $min) { throw "only $rows row(s) in $table, expected $min or more" }
    "restore test ok: $($latest.Name), $rows rows in $table"
}
finally { Remove-Item $tmp -Recurse -Force }
```

Python کد اجراشده و تست‌شده در /examples examples/ops/restore/restore_test.py

```python
# From the copy you would have after losing the server, not from the original.
expected = (copy / "SHA256SUMS").read_text().split()[0]
if hashlib.sha256((copy / "app.db.gz").read_bytes()).hexdigest() != expected:
    sys.exit(f"checksum mismatch in {copy}")
with tempfile.TemporaryDirectory() as tmp:
    restored = Path(tmp) / "restored.db"
    with gzip.open(copy / "app.db.gz", "rb") as z, open(restored, "wb") as f:
        shutil.copyfileobj(z, f)
    with closing(sqlite3.connect(restored)) as conn:
        check = conn.execute("PRAGMA integrity_check").fetchone()[0]
        if check != "ok":
            sys.exit(f"integrity check: {check}")
        # A backup of an empty database restores perfectly; it is still no backup.
        rows = conn.execute(f"SELECT count(*) FROM {table}").fetchone()[0]
if rows < minimum:
    sys.exit(f"only {rows} row(s) in {table}, expected {minimum} or more")
print(f"restore test ok: {latest.name}, {rows} rows in {table}")
```

restore یک .bak در SQL Server لینوکسی (restore-db.sh)

همه‌ی زبان‌ها از کد واقعی پروژه (کوتاه‌شده)

```
# One MOVE per file in the backup: Windows paths mean nothing here.
while IFS='|' read -r logical _ type _; do
  [[ -z "$logical" || -z "$type" ]] && continue
  case "$type" in L) ext=ldf ;; *) ext=mdf ;; esac
  [[ $n -gt 0 && "$ext" == mdf ]] && ext=ndf
  moves+=", MOVE N'$logical' TO N'/var/opt/mssql/data/${db}_$n.$ext'"
  n=$((n + 1))
done < <(sql "SET NOCOUNT ON; RESTORE FILELISTONLY FROM DISK = N'$inside'")

sql "IF DB_ID(N'$db') IS NOT NULL ALTER DATABASE [$db] SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
     RESTORE DATABASE [$db] FROM DISK = N'$inside' WITH REPLACE, RECOVERY$moves;
     ALTER DATABASE [$db] SET MULTI_USER;"

# Logins live in master, not in the database: drop the orphaned users and recreate
# them with the passwords from .env.
sql "USE [LabFlow]; DROP USER IF EXISTS labflow_svc;"
run_sql_file deploy/labflow/sql-login.sql -v "LABFLOW_DB_PASSWORD=$(env_val LABFLOW_DB_PASSWORD .env)"
```

اشتباه‌های رایج

همه **بکاپ فقط روی همان سرور**

چرا: وضعیت فعلی ۱۵ :VPS بکاپ سالم که با یک خرابی دیسک یا حذف VPS همه با هم از بین می‌روند.

درست: کپی شبانه با rclone یا restic به ارائه‌دهنده‌ی دیگر، بعد از backup.sh در همان cron.

همه **کلید مقصد بیرونی با اجازه‌ی پاک کردن**

چرا: کسی که سرور را گرفته با همان کلید بکاپ‌های بیرونی را هم پاک یا رمزگذاری می‌کند (باج‌افزار دقیقاً همین را می‌کند).

درست: کلید فقط-نوشتن (B2: application key بدون deleteFiles؛ S3: Object Lock)، یا pull از بیرون به‌جای push.

همه **«بکاپ گرفته شد» به‌جای «restore شد»**

چرا: فایل .gz ناقص، فایل خالی، یا بکاپ دیتابیسی که مدت‌هاست خالی است، همه «موفق» گزارش می‌شوند.

درست: تست restore خودکار هر شب، با شمردن ردیف‌های یک جدول واقعی.

همه **فراموش کردن .env**

چرا: بدون رمز sa و رمزهای سرویس‌ها، دیتابیس restoreشده login ندارد و برنامه‌ها وصل نمی‌شوند.

درست: .env در password manager، و در تمرین restore از آن استفاده کن.

Bash **cp -r به پوشه‌ای که از قبل هست**

چرا: cp -r a b وقتی b هست، b/a می‌سازد؛ restore بعدی فایل را در مسیر اشتباه پیدا نمی‌کند.

درست: اول چک کن پوشه نیست، یا cp -rT.

این کد کجای پروژه‌هاست

`ControlPanel/deploy/vps/README.md:169`

«Off-server copy: these backups die with the server»

`ControlPanel/deploy/vps/backup.sh:8`

در برابر از دست دادن سرور محافظت نمی‌کند

`ControlPanel/deploy/vps/restore-db.sh:12`

loginها در master هستند؛ کاربرهای restoreشده یتیم‌اند

`ControlPanel/deploy/vps/restore-db.sh:51`

یک MOVE برای هر فایل داخل بکاپ

تمرین

▼ 1. به خط crontab فعلی چه اضافه می‌کنی تا کپی بیرونی و تست restore هم هر شب اجرا شوند؟

پاسخ: 30 3 * * * cd /srv/portfolio/ControlPanel && deploy/vps/backup.sh && deploy/vps/offsite.sh && deploy/vps/restore-test.sh >> /srv/backups/backup.log 2>&1. با && هر قدم فقط اگر قبلی موفق بود اجرا می‌شود. (offsite.sh و restore-test.sh هنوز روی سرور نیستند.)

▼ 2. در تست‌ها، برای فایل آسیب‌دیده کدام قدم شکست می‌خورد و چرا قبل از gunzip؟

پاسخ: checksum. به انتهای app.db.gz چند بایت اضافه شده، پس SHA-256 فرق می‌کند. چک checksum قبل از باز کردن، خرابی را با پیام روشن می‌گوید، نه با خطای مبهم gzip.

▼ 3. تمرین restore کامل روی سرور: LabFlow را از بکاپ دیشب به نام دیگری restore کن بدون اینکه LabFlow اصلی را دست بزنی.

پاسخ: فایل را LabFlowRestoreTest.bak.gz نام‌گذاری کن و restore-db.sh را رویش اجرا کن؛ نام دیتابیس از نام فایل می‌آید. بعد SELECT COUNT(*) روی یک جدول، مقایسه با اصل، و DROP DATABASE LabFlowRestoreTest.

جمله‌ی مصاحبه

Every night the newest backup is copied to another provider with a write-only key, and then restored from that copy into a scratch database and checked: checksum, integrity, and real rows, because the only backup that counts is one I have actually restored.

درس 7 · ops

CI/CD to a VPS: deploy after tests, with a key that can only deploy

CI/CD: استقرار بعد از تست، با کلیدی که فقط deploy بلد است

secrets | known_hosts | authorized_keys | forced command | concurrency | workflow_run

بعد از این درس می‌توانی

- استقرار را به نتیجه‌ی CI ببندی: فقط وقتی تست‌ها روی main سبز شدند، و دستی وقتی لازم است.
- به CI یک کلید SSH بدهی که روی سرور فقط یک دستور را اجرا می‌کند، و host key سرور را از قبل ثابت کنی.
- همین را در Azure Pipelines هم بشناسی: trigger از pipeline دیگر، environment، و secretها.

سناریو از پروژه‌ی واقعی

deploy-vps.yml در Control Panel با workflow_run بعد از تمام شدن CI اجرا می‌شود و فقط اگر نتیجه success باشد ادامه می‌دهد. خود workflow هیچ دستوری به سرور نمی‌فرستد: کلید GitHub در authorized_keys با "restrict,command="...deploy.sh" ثبت شده، پس هر چه درخواست شود فقط deploy.sh اجرا می‌شود. host key هم از secret VPS_KNOWN_HOSTS می‌آید، نه از اولین جوابی که می‌رسد. تست این درس یک sshd واقعی در کانتینر بالا می‌آورد و ثابت می‌کند: دستور دیگر فقط deploy می‌کند، port forwarding رد می‌شود، و سروری با کلید دیگر پذیرفته نمی‌شود. LabFlow هم CI خود را در Azure Pipelines دارد؛ ستون دوم همان استقرار را آنجا نشان می‌دهد.

سؤال چرا workflow جدا با workflow_run و نه یک job دیگر در همان CI؟

پاسخ CI روی هر push و PR اجرا می‌شود و نباید به secretهای سرور دسترسی داشته باشد. workflow جدا permissions: {} دارد، secretها را فقط خودش می‌بیند، و می‌شود دستی هم اجرایش کرد.

سؤال اگر کلید CI دزدیده شود چه؟

پاسخ با command= و restrict، دزد فقط می‌تواند deploy.sh را اجرا کند: همان کد GitHub را دوباره بکشد و بسازد. shell، کپی فایل و tunnel ندارد. بعد کلید را از authorized_keys پاک می‌کنی.

مفهوم‌ها

trigger بعد از CI

workflow_run با types: [completed] روی هر نتیجه‌ای اجرا می‌شود، حتی failure؛ شرط if روی 'github.event.workflow_run.conclusion == 'success لازم است. نکته‌ی پنهان: workflow_run همیشه نسخه‌ی workflow روی شاخه‌ی پیش‌فرض را اجرا می‌کند، نه نسخه‌ی شاخه‌ی pushشده. در Azure معادلش resources.pipelines با trigger است.

صف، نه لغو

concurrency با cancel-in-progress: false یعنی deploy دوم منتظر اولی می‌ماند. لغو وسط docker compose up -- build سرور را در وضعیت نیمه رها می‌کند. روی خود سرور هم flock همین کار را می‌کند؛ دو لایه، چون deploy از release.ps1 هم می‌آید. در Azure: environment با Exclusive lock.

forced command

یک خط در ~/.ssh/authorized_keys: restrict,command="/path/deploy.sh" ssh-ed25519 AAAA.... همه‌چیز را می‌بندد (agent، port و X11 forwarding، و pty). sshd درخواست اصلی را در SSH_ORIGINAL_COMMAND می‌گذارد؛ اسکریپت می‌تواند برای کارهای محدود مثل «فقط ControlPanel» از آن استفاده کند، با اعتبارسنجی سخت.

host key ثابت

StrictHostKeyChecking=accept-new یا ssh-keyscan داخل workflow یعنی به هر کسی که اول جواب داد اعتماد کن. به‌جایش known_hosts در secret، که یک بار یک آدم fingerprint آن را با خروجی bootstrap.sh یا کنسول ارائه‌دهنده مقایسه کرده. اگر سرور عوض شد، ssh با «Host key verification failed» متوقف می‌شود، که همان رفتار درست است.

مقایسه

اجرا بعد از CI موفق	
GitHub Actions	`on: workflow_run (completed) + if: conclusion == 'success'`
Azure Pipelines	`resources.pipelines: trigger` روی pipeline CI

Azure فقط روی اجرای موفق trigger می‌کند؛ GitHub روی هر نتیجه و شرط با توست.

اجرای دستی	
GitHub Actions	`workflow_dispatch`
Azure Pipelines	`Run pipeline (همیشه هست)`

در GitHub بدون workflow_dispatch دکمه‌ای در Actions نیست.

یکی‌یکی	
GitHub Actions	`concurrency: { group, cancel-in-progress: false }`
Azure Pipelines	`environment` با `Exclusive lock`

در هر دو، قفل CI جای flock سرور را نمی‌گیرد؛ deploy از جاهای دیگر هم می‌آید.

secretها	
GitHub Actions	`${{ secrets.VPS_SSH_KEY }}` در `env`
Azure Pipelines	`Secure file + variable group`، با `env:` صریح

متغیر secret در Azure خودکار env نمی‌شود؛ باید در env: نامش را بیاوری.

تأیید انسانی	
GitHub Actions	`environment: production` با `required reviewers`
Azure Pipelines	`Approvals and checks` روی `environment`

برای یک VPS شخصی لازم نیست؛ برای production تیمی معمول است.

دسترسی توکن	
GitHub Actions	`permissions: {}`
Azure Pipelines	`Limit job authorization scope (تنظیم پروژه)`

در GitHub در خود فایل، در Azure در تنظیمات پروژه.

کد

کِی اجرا شود

GitHub Actions کد اجراشده و تست‌شده در /examples `examples/ops/ci/deploy-vps.yml`

```yaml
on:
  # Runs when another workflow finishes, whatever its result: the job checks it.
  workflow_run:
    workflows: [CI]
    types: [completed]
    branches: [main]
  workflow_dispatch:

# Deploys queue up; one is never cancelled halfway through a rebuild.
concurrency:
  group: deploy-vps
  cancel-in-progress: false

# The job needs no GitHub token rights: it only talks to the server.
permissions: {}
```

Azure Pipelines معادل نوشته‌شده، اجرا نشده

```yaml
# Runs after the CI pipeline succeeds on main; never on its own pushes.
trigger: none
pr: none

resources:
  pipelines:
    - pipeline: ci
      source: LabFlow-CI          # the pipeline made from azure-pipelines.yml
      trigger:
        branches:
          include: [main]
```

اجرای deploy روی سرور

GitHub Actions کد اجراشده و تست‌شده در /examples `examples/ops/ci/deploy-vps.yml`

```yaml
if: >-
  vars.VPS_DEPLOY_ENABLED == 'true' &&
  (github.event_name == 'workflow_dispatch' || github.event.workflow_run.conclusion == 'success')
runs-on: ubuntu-24.04
timeout-minutes: 30
steps:
  - name: Run deploy.sh on the server
    env:
      VPS_HOST: ${{ secrets.VPS_HOST }}
      VPS_SSH_KEY: ${{ secrets.VPS_SSH_KEY }}
      VPS_KNOWN_HOSTS: ${{ secrets.VPS_KNOWN_HOSTS }}
    run: |
      install -m 700 -d ~/.ssh
      printf '%s\n' "$VPS_SSH_KEY" > ~/.ssh/id_ed25519
      chmod 600 ~/.ssh/id_ed25519
      # Pinned host key: never trust whatever answers on the first connect.
      printf '%s\n' "$VPS_KNOWN_HOSTS" > ~/.ssh/known_hosts
      # The key may only run deploy.sh (command= in authorized_keys): no command here.
      ssh -T -o BatchMode=yes "deploy@$VPS_HOST"
```

Azure Pipelines معادل نوشته‌شده، اجرا نشده

```
stages:
  - stage: Deploy
    jobs:
      - deployment: Vps
        # Approvals and the exclusive lock are set on the environment, in the portal.
        environment: production
        pool:
          vmImage: ubuntu-24.04
        variables:
          - group: vps            # VPS_HOST, VPS_KNOWN_HOSTS (secret)
        strategy:
          runOnce:
            deploy:
              steps:
                - task: DownloadSecureFile@1
                  name: key
                  inputs:
                    secureFile: vps-deploy-key
                - script: |
                    install -m 700 -d ~/.ssh
                    install -m 600 "$(key.secureFilePath)" ~/.ssh/id_ed25519
                    printf '%s\n' "$VPS_KNOWN_HOSTS" > ~/.ssh/known_hosts
                    ssh -T -o BatchMode=yes "deploy@$VPS_HOST"
                  env:
                    # Secret variables reach a script only when named here.
                    VPS_HOST: $(VPS_HOST)
                    VPS_KNOWN_HOSTS: $(VPS_KNOWN_HOSTS)
```

سمت سرور: کلیدی که فقط deploy می‌کند (تست‌شده با sshd واقعی)

همه‌ی زبان‌ها کد اجراشده و تست‌شده در /examples examples/ops/tests/run.sh

```
# One line in the server's ~deploy/.ssh/authorized_keys. Whatever the client asks
# for, sshd runs only this command; restrict also turns off port forwarding, agent
# forwarding, X11 and the terminal. A stolen CI key can deploy, and nothing else.
echo "restrict,command=\"/usr/local/bin/only-deploy\" $(cat /tmp/ci/key.pub)" >
/home/deployer/.ssh/authorized_keys
```

host key ثابت به‌جای اعتماد به اولین جواب

همه‌ی زبان‌ها کد اجراشده و تست‌شده در /examples examples/ops/tests/run.sh

```
# The fingerprint is compared once, by a person (bootstrap.sh prints it); from then on
# CI trusts exactly this key, and a server that answers with another one is refused.
ssh-keyscan -p 2222 -t ed25519 localhost 2>/dev/null > /tmp/ci/known_hosts
ssh-keygen -lf /tmp/ci/known_hosts
```

اشتباه‌های رایج

GitHub Actions **workflow_run بدون شرط conclusion**

چرا: types: [completed] روی CI شکست‌خورده هم اجرا می‌شود و کد خراب را deploy می‌کند.

درست: if: github.event.workflow_run.conclusion == 'success'.

GitHub Actions **ویرایش workflow_run روی یک شاخه و انتظار اثر**

چرا: GitHub نسخه‌ی شاخه‌ی پیش‌فرض را اجرا می‌کند؛ تغییر تا merge در main دیده نمی‌شود.

درست: تغییرهای deploy-vps.yml را روی main امتحان کن، با workflow_dispatch.

همه **ssh-keyscan یا StrictHostKeyChecking=no در CI**

چرا: هر کسی بین runner و سرور بنشیند (DNS، IP جابه‌جاشده) کلید CI و دستور را می‌گیرد.

درست: known_hosts در secret، بعد از مقایسه‌ی fingerprint با چشم.

همه **کلید CI بدون restrict**

چرا: command= به‌تنهایی port forwarding را باز می‌گذارد؛ با کلید دزدیده می‌شود از سرور به شبکه‌ی داخلی Docker تونل زد.

درست: restrict,command="..."؛ تست این درس ثابت می‌کند -R رد می‌شود.

Azure Pipelines **secret variable که در script خالی است**

چرا: Azure متغیرهای secret را خودکار به محیط script نمی‌دهد.

درست: در env: نامشان را صریح بیاور.

این کد کجای پروژه‌هاست

`ControlPanel/.github/workflows/deploy-vps.yml:8`

workflow_run بعد از CI روی main

`ControlPanel/.github/workflows/deploy-vps.yml:14`

concurrency بدون لغو

`ControlPanel/.github/workflows/deploy-vps.yml:38`

known_hosts از secret، نه از اولین اتصال

`ControlPanel/deploy/vps/README.md:143`

"restrict,command=".../deploy.sh در authorized_keys

`LabFlow/azure-pipelines.yml:8`

CI در Azure Pipelines: trigger و stages

تمرین

▼ 1. با کلید CI از PC خودت 'ssh deploy@server 'cat .env بزن. چه چیزی برمی‌گردد؟

پاسخ: خروجی deploy.sh: pull، build و health. cat هرگز اجرا نمی‌شود؛ درخواست فقط در SSH_ORIGINAL_COMMAND می‌ماند. تست examples/ops همین را با cat /etc/shadow نشان می‌دهد.

▼ 2. سرور را دوباره نصب کردی و host key عوض شد. Action چه می‌کند و چه باید بکنی؟

پاسخ: با «Host key verification failed» قرمز می‌شود. fingerprint جدید را از خروجی bootstrap.sh یا کنسول ارائه‌دهنده با ssh-keygen -lf روی خروجی ssh-keyscan مقایسه کن و بعد VPS_KNOWN_HOSTS را به‌روز کن.

▼ 3. push به LedgerDashboard هیچ deployی راه نمی‌اندازد. دو راه برای درست کردنش بگو.

پاسخ: ۱) دستی: Actions → Deploy to VPS → Run workflow (همان کاری که README می‌گوید). ۲) در CI آن repo یک قدم repository_dispatch به ControlPanel با یک token محدود، و در deploy-vps.yml یک on: repository_dispatch.

جمله‌ی مصاحبه

The deploy workflow runs only after CI succeeds on main, queues instead of cancelling, and uses an SSH key the server restricts to a single forced command, against a host key pinned in a secret, so a stolen key can deploy and nothing else.

درس 8 · ops

A reverse proxy in front of every app

reverse proxy: HTTPS خودکار، basic auth و هدرهای امنیتی

reverse proxy | automatic HTTPS | basic auth | security headers | HSTS

بعد از این درس می‌توانی

- همه‌ی برنامه‌ها را پشت یک reverse proxy بگذاری که تنها چیز رو به اینترنت است.
- HTTPS را با Caddy خودکار بگیری و بدانی nginx برای همین کار چه چیزی کم دارد.
- سایت‌های خصوصی را با basic auth ببندی و هدرهای امنیتی را روی همه‌ی پاسخ‌ها، حتی ۴۰۱، بگذاری.

سناریو از پروژه‌ی واقعی

روی VPS فقط Caddy پورت 80 و 443 دارد. هفت دامنه، هر کدام یک reverse_proxy به یک کانتینر: panel، rebiomed، code، english و report عمومی‌اند و ledger، labflow و ctx پشت basic auth. یک snippet به نام (common) فشرده‌سازی، nosniff، HSTS و Referrer-Policy را می‌گذارد و هدر Server را برمی‌دارد، و (private) رمز را. گواهی Let's Encrypt هر دامنه را Caddy خودش می‌گیرد و تمدید می‌کند. ReBiomed داخل کانتینر خودش هم یک nginx دارد که فایل‌های React را می‌دهد و /api را به Node می‌فرستد. این درس همان دو سایت را با هر دو ابزار می‌سازد و با curl چک می‌کند چه چیزی به مرورگر می‌رسد.

سؤال چرا هر برنامه پورت خودش را باز نکند؟

پاسخ یک جا برای HTTPS، یک جا برای رمز، یک جا برای هدرها و لاگ. و چون Docker از ufw رد می‌شود، هر پورت publishشده واقعاً به اینترنت باز است. پشت proxy، دیتابیس‌ها و APIها هیچ پورتی ندارند.

سؤال HSTS روی پاسخ ۴۰۱ چه فایده‌ای دارد؟

پاسخ اولین چیزی که مرورگر از سایت خصوصی می‌بیند ۴۰۱ است. اگر HSTS آنجا نباشد، مرورگر یاد نمی‌گیرد دفعه‌ی بعد مستقیم HTTPS بزند.

مفهوم‌ها

HTTPS خودکار

در Caddy آدرس سایت دامنه است (panel.example.com {) و همین کافی است: گواهی از Let's Encrypt، تمدید، و redirect از HTTP. شرطش این است که DNS از قبل به سرور اشاره کند و 80 و 443 باز باشند. nginx گواهی نمی‌گیرد؛ certbot جدا، یک timer برای تمدید، و reload بعد از هر تمدید لازم است.

basic auth

Caddy hash bcrypt می‌خواهد (caddy hash-password) و nginx فایل htpasswd (openssl passwd -apr1 یا htpasswd). رمز روی HTTPS امن منتقل می‌شود ولی در هر درخواست فرستاده می‌شود؛ برای یک صاحب و چند مهمان خوب است، برای کاربران واقعی نه. در IIS معادل مستقیمی نیست: Basic Authentication با حساب‌های ویندوز کار می‌کند.

هدرها

Strict-Transport-Security: فقط HTTPS برای یک سال. X-Content-Type-Options: nosniff: مرورگر نوع فایل را حدس نزند. Referrer-Policy: آدرس کامل به سایت‌های دیگر نرود. در nginx نکته‌ی پنهان دو تاست: بدون always هدر روی ۴xx و ۵xx نمی‌آید، و هر location که add_header خودش را داشته باشد هیچ‌کدام از هدرهای بالا را نمی‌گیرد. در Caddy یکی: اگر یک بلوک header حذف (-Server) داشته باشد، کل بلوک deferred می‌شود، و هدرهای deferred روی پاسخ خطا مثل ۴۰۱ basic_auth از دست می‌روند. تست این درس این را در Caddyfile سرور پیدا کرد و آنجا هم با دو بلوک header درست شد.

اعتماد به هدرهای proxy

برنامه‌ی پشت proxy درخواست را از proxy می‌بیند، نه از کاربر. X-Forwarded-For و X-Forwarded-Proto آدرس و پروتکل اصلی را می‌برند؛ Caddy خودش می‌فرستد و در nginx باید proxy_set_header بنویسی. برنامه (UseForwardedHeaders در ASP.NET Core، trust proxy در Express) باید فقط به proxy اعتماد کند.

مقایسه

HTTPS	
Caddy	`panel.example.com { ... }` (خودکار)
nginx	`certbot --nginx` + timer تمدید

Caddy redirect از HTTP و تمدید را هم خودش می‌کند.

reverse proxy	
Caddy	`reverse_proxy app:80`
nginx	`location / { proxy_pass http://app:80; proxy_set_header ... }`

Caddy هدرهای X-Forwarded-* را خودش می‌فرستد؛ nginx فقط اگر بنویسی.

تکه‌ی تکرارشونده

Caddy	`(common) { ... }` و `import common`
nginx	`include snippets/common.conf;`

snippet در Caddy داخل همان فایل است؛ در nginx فایل جدا.

basic auth

Caddy	`basic_auth { user $2a$... }`
nginx	`auth_basic "private"; auth_basic_user_file htpasswd;`

hash bcrypt پر از $ است؛ در .env یا compose بدون نقل‌قول، Compose آن را متغیر می‌خواند.

حذف هدر Server

Caddy	`header -Server`
nginx	`server_tokens off` (فقط نسخه را برمی‌دارد)

nginx بدون ماژول headers-more «Server: nginx» را همیشه می‌فرستد. تست هر دو را ثابت می‌کند.

هدر روی ۴۰۱

Caddy	`header -Server` جدا از (نه deferred)
nginx	`add_header ... always`

هر دو پیش‌فرض ۴۰۱ را بدون هدر می‌فرستند، به دو دلیل مختلف. تست هر دو را ثابت می‌کند.

کد

دو سایت: عمومی و خصوصی

Caddy کد اجراشده و تست‌شده در /examples `examples/ops/proxy/Caddyfile`

```
(common) {
	encode zstd gzip
	# Two blocks on purpose. A deletion (-Server) defers its whole block until the
	# final response, and deferred headers are lost when a handler fails, as basic_auth
	# does with its 401. Set right away, these reach the 401 too.
	header {
		Strict-Transport-Security "max-age=31536000"
		X-Content-Type-Options nosniff
		Referrer-Policy strict-origin-when-cross-origin
	}
	# Deferred: removes the app's own Server header too.
	header -Server
}

(private) {
	basic_auth {
		# A bcrypt hash (caddy hash-password), never the password itself.
		{$BASIC_AUTH_USER} {$BASIC_AUTH_HASH}
	}
}

:8080 {
	import common
	reverse_proxy app:80
}

:8081 {
	import common
	import private
	reverse_proxy app:80
}
```

nginx کد اجراشده و تست‌شده در /examples examples/ops/proxy/nginx.conf

```
server_tokens off;   # hides the version only; "Server: nginx" stays
gzip on;
# always: also on 401 and 5xx (without it nginx adds headers to 2xx and 3xx only).
# A location or server with its own add_header gets none of these: they are not merged.
add_header Strict-Transport-Security "max-age=31536000" always;
add_header X-Content-Type-Options nosniff always;
add_header Referrer-Policy strict-origin-when-cross-origin always;

server {
  listen 8090;
  location / {
    proxy_pass http://app:80;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}

server {
  listen 8091;
  auth_basic "private";
  auth_basic_user_file /etc/nginx/htpasswd;
  location / {
    proxy_pass http://app:80;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

آنچه مرورگر می‌گیرد، با curl

همه‌ی زبان‌ها کد اجراشده و تست‌شده در examples/ examples/ops/proxy/test.sh

```
#!/usr/bin/env bash
# Starts the app behind Caddy and behind nginx (compose.yml) and checks what a browser
# gets back: status codes, basic auth and the security headers. Needs Docker and curl.
set -euo pipefail
cd "$(dirname "$0")"

tmp="$(mktemp -d)"
trap 'docker compose down --volumes --remove-orphans >/dev/null 2>&1; rm -rf "$tmp"' EXIT

export BASIC_AUTH_USER=arash
BASIC_AUTH_HASH="$(docker run --rm caddy:2.10-alpine caddy hash-password --plaintext secret)"
export BASIC_AUTH_HASH
printf 'arash:%s\n' "$(openssl passwd -apr1 secret)" > "$tmp/htpasswd"
chmod 644 "$tmp/htpasswd"
export HTPASSWD="$tmp/htpasswd"

docker compose up -d --wait

fail=0
status() { curl -s -o /dev/null -w '%{http_code}' -m 5 "$@"; }
header() { local name="$1"; shift; { curl -s -D - -o /dev/null -m 5 "$@" | tr -d '\r' | grep -i "^$name:" | cut
-d' ' -f2-; } || true; }
expect() {
  if [[ "$2" == "$3" ]]; then echo "  ok    $1"; else echo "  FAIL  $1: got '$2', expected '$3'"; fail=1; fi
}

for _ in $(seq 30); do [[ "$(status http://localhost:18080/)" == 200 && "$(status http://localhost:18090/)" ==
200 ]] && break; sleep 1; done

echo "Caddy"
expect "public site answers" "$(status http://localhost:18080/)" 200
expect "app's Server header removed" "$(header Server http://localhost:18080/)" ""
expect "HSTS" "$(header Strict-Transport-Security http://localhost:18080/)" "max-age=31536000"
expect "nosniff" "$(header X-Content-Type-Options http://localhost:18080/)" "nosniff"
expect "private site without password" "$(status http://localhost:18081/)" 401
expect "private site, wrong password" "$(status -u arash:wrong http://localhost:18081/)" 401
expect "private site, right password" "$(status -u arash:secret http://localhost:18081/)" 200
expect "HSTS on the 401 too" "$(header Strict-Transport-Security http://localhost:18081/)" "max-age=31536000"

echo "nginx"
expect "public site answers" "$(status http://localhost:18090/)" 200
expect "Server header: name without version" "$(header Server http://localhost:18090/)" "nginx"
expect "HSTS" "$(header Strict-Transport-Security http://localhost:18090/)" "max-age=31536000"
expect "private site without password" "$(status http://localhost:18091/)" 401
expect "private site, right password" "$(status -u arash:secret http://localhost:18091/)" 200
expect "HSTS on the 401 too (always)" "$(header Strict-Transport-Security http://localhost:18091/)" "max-
age=31536000"

exit "$fail"
```

اشتباه‌های رایج

Caddy **هدرهای امنیتی و -Server در یک بلوک header**

چرا: حذف کل بلوک را deferred می‌کند و هدرهای deferred روی ۴۰۱ basic_auth نمی‌آیند. Caddyfile سرور همین شکل را داشت: ledger ،labflow و ctx اولین پاسخشان را بدون HSTS می‌دادند (حالا درست شده).

درست: یک بلوک header برای تنظیم‌ها و یک header -Server جدا؛ تست proxy/test.sh هر دو را چک می‌کند.

Caddy **hash bcrypt در .env بدون نقل‌قول تکی**

چرا: Compose هر $ را شروع یک متغیر می‌داند؛ hash بریده می‌شود و هیچ رمزی کار نمی‌کند.

درست: در .env با '...' (کاری که make-env.sh می‌کند)، یا در compose با $$.

Caddy **دامنه‌ای که هنوز DNS ندارد در Caddyfile**

چرا: Caddy گواهی نمی‌گیرد و مدام دوباره تلاش می‌کند؛ Let's Encrypt برای تلاش‌های ناموفق محدودیت دارد.

درست: اول رکورد A، بعد Caddyfile. برای آزمایش، acme_ca staging.

nginx **add_header بدون always**

چرا: صفحه‌ی ۴۰۱ و ۵۰۲ بدون HSTS و nosniff می‌رسد.

درست: always در پایان هر add_header.

nginx **یک add_header در location**

چرا: همه‌ی add_headerهای سطح server و http برای آن location حذف می‌شوند؛ ارث‌بری ادغامی نیست.

درست: هدرها را در یک فایل include کن و در هر location که add_header دارد دوباره include کن.

همه **سایت خصوصی که فقط «مخفی» است**

چرا: نام دامنه‌ها در Certificate Transparency عمومی ثبت می‌شوند؛ ربات‌ها چند دقیقه بعد از صدور گواهی سر می‌زنند.

درست: رمز (basic auth یا login واقعی)، نه آدرس مخفی.

این کد کجای پروژه‌هاست

`ControlPanel/deploy/vps/Caddyfile:8`

(common): فشرده‌سازی، nosniff ،HSTS، و حذف Server در بلوک جدا، تا هدرها روی ۴۰۱ هم بیایند

`ControlPanel/deploy/vps/Caddyfile:28`

(private): basic auth برای LedgerDashboard ،LabFlow و ctx

`ControlPanel/deploy/vps/make-env.sh:63`

hash بین نقل‌قول تکی در .env تا Compose $ها را نخواند

`ControlPanel/deploy/rebiomed/nginx.conf:8`

تمرین

▼ 1. <curl -I https://panel.<domain> را بزن. کدام هدرهای (common) را می‌بینی و کدام را نه؟

پاسخ: x-content-type-options، strict-transport-security و referrer-policy هست؛ server نیست. اگر content-encoding ندیدی، چون -I درخواست HEAD است و Accept-Encoding نفرستاده؛ با --compressed دوباره امتحان کن.

▼ 2. در nginx.conf این درس، داخل location / یک add_header X-Frame-Options DENY اضافه کن. تست کدام خط‌ها را قرمز می‌کند؟

پاسخ: HSTS روی 8090 دیگر نمی‌آید، چون آن location حالا add_header خودش را دارد و هیچ‌کدام از سطح بالا را به ارث نمی‌برد. سایت 8091 (location خودش بدون add_header) سالم می‌ماند.

▼ 3. یک سایت دیگر به VPS اضافه کن، مثلاً <status.<domain> که فقط برای تو باشد. چه چیزهایی باید عوض شوند؟

پاسخ: رکورد A در DNS، یک بلوک در Caddyfile با import common و import private و reverse_proxy، متغیر دامنه در compose.vps.yml و .env، و deploy.sh (که Caddyfile تغییرکرده را می‌بیند و caddy را restart می‌کند).

جمله‌ی مصاحبه

Only the reverse proxy faces the internet: Caddy gets and renews every certificate itself, adds the security headers to every response including 401s, and puts the private sites behind basic auth, while nginx needs certbot for the certificates and the always flag for the headers.